第8章

# 创建型设计模式

陈龙彪
longbiaochen@xmu.edu.cn

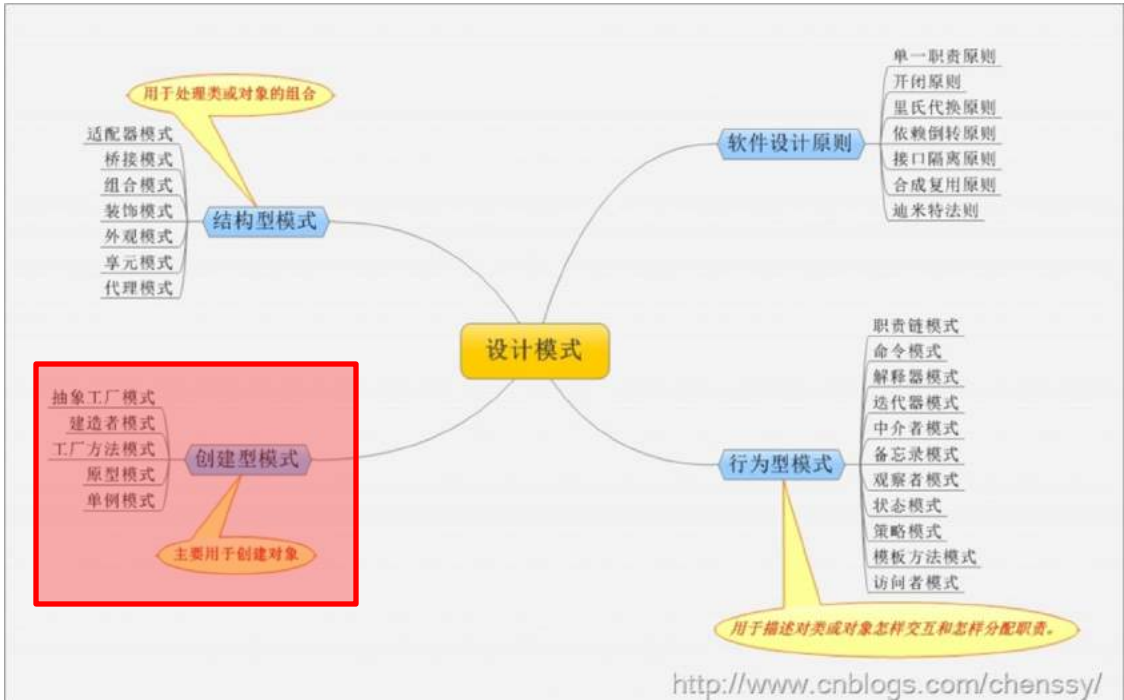
厦门大学信息学院
2019年4月

# 设计模式

| 范围\目的 | 创建型模式 | 结构型模式 | 行为型模式 |
| --- | --- | --- | --- |
| **类模式** | 工厂方法模式 | （类）适配器模式 | 解释器模式<br>模板方法模式 |
| **对象模式** | 抽象工厂模式<br>建造者模式<br>原型模式<br>单例模式 | （对象）适配器模式<br>桥接模式<br>组合模式<br>装饰模式<br>外观模式<br>享元模式<br>代理模式 | 职责链模式<br>命令模式<br>迭代器模式<br>中介者模式<br>备忘录模式<br>观察者模式<br>状态模式<br>策略模式<br>访问者模式 |

# 本章教学内容

- 创建型设计模式
  - 简单工厂模式
  - 工厂方法模式
  - 抽象工厂模式
  - 建造者模式
  - 原型模式
  - 单例模式



# 创建型模式

- 概述
  - 创建型模式（Creational Pattern）对类的实例化过程进行抽象，将软件模块中对象的创建和对象的使用分离
  - 为了使软件的结构更加清晰，外界对于这些对象只需要知道它们共同的接口，而不清楚其具体的实现细节，使整个系统的设计更加符合单一职责原则

# 创建型模式

- 概述
  - 创建型模式在创建什么（What），由谁创建（Who），何时创建（When）等方面都为软件设计者提供了尽可能大的灵活性
  - 创建型模式隐藏了类的实例的创建细节，通过隐藏对象如何被创建和组合在一起达到使整个系统独立的目的

# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）

# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）



# 简单工厂模式

- 举个栗子



想吃苹果？

# 简单工厂模式

- 举个栗子

**获取苹果的两种方式**

**自己种苹果树**

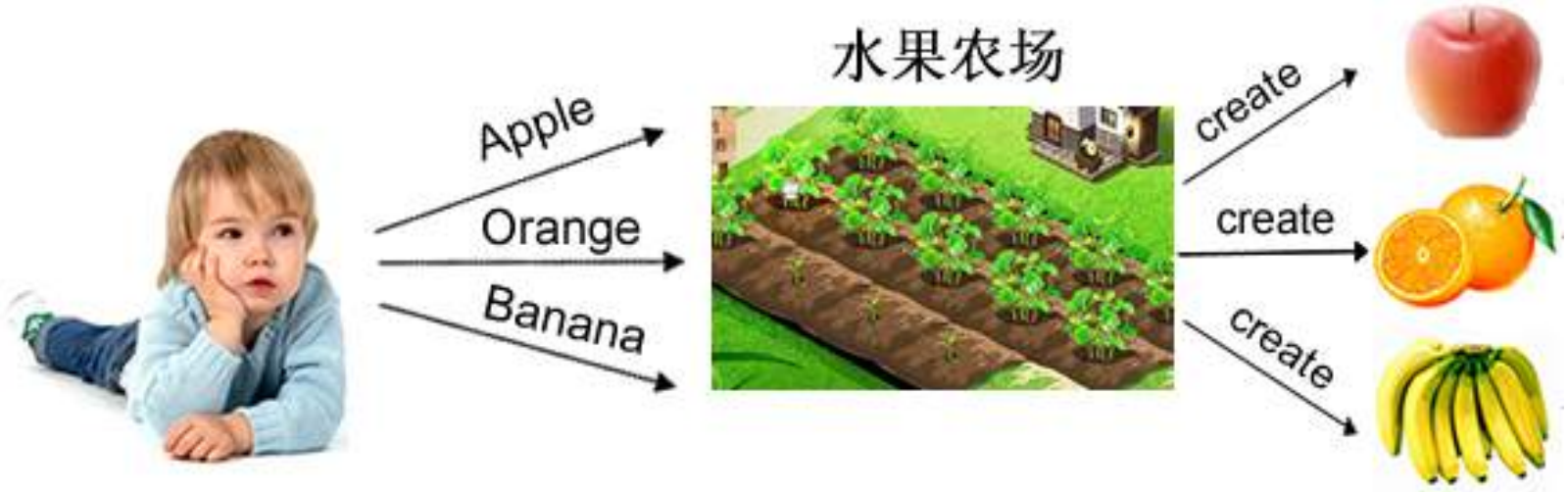


**去超市买**



# 简单工厂模式

- 模式动机
  - 只需要知道水果的名字则可得到相应的水果



# 简单工厂模式

- 模式动机
  - 考虑一个简单的软件应用场景：一个软件系统可以提供多个外观不同的按钮（如圆形按钮、矩形按钮、菱形按钮等），这些按钮都源自同一个基类，不过在继承基类后不同的子类修改了部分属性从而使得它们可以呈现不同的外观，如果我们希望在使用这些按钮时，不需要知道这些具体按钮类的名字，只需要知道表示该按钮类的一个参数，并提供一个调用方便的方法，把该参数传入方法即可返回一个相应的按钮对象，此时，就可以使用简单工厂模式

```
Button btn = new RoundButton();
Button btn = ButtonFactory.createButton('round');
```

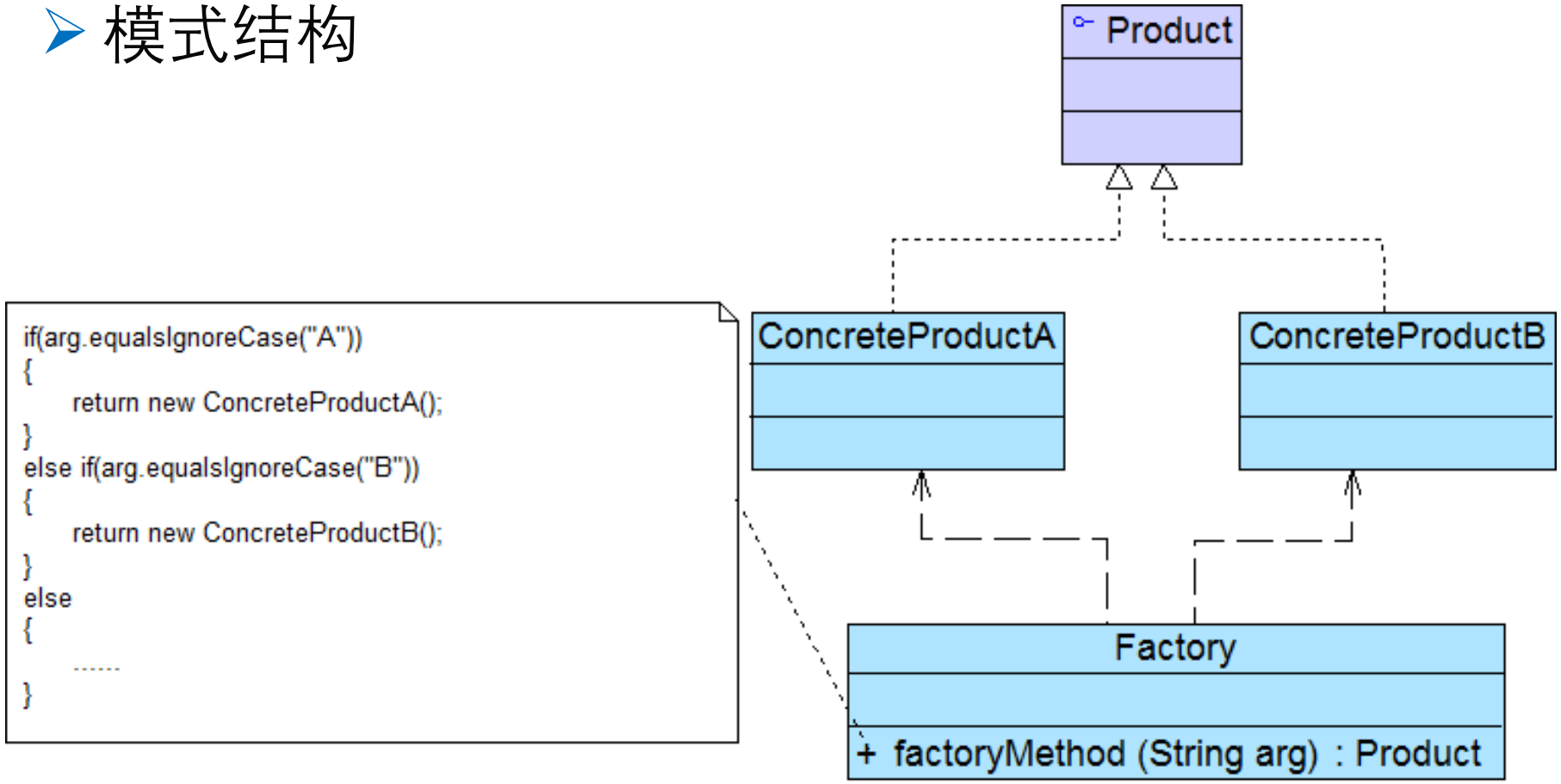
# 简单工厂模式

- 定义
  - 简单工厂模式（Simple Factory Pattern）：又称为静态工厂方法（Static Factory Method）模式。可以根据参数的不同返回不同类的实例
  - 简单工厂模式专门定义一个类来负责创建其他类的实
  - 被创建的实例通常都具有共同的父类

```
Button btn = new RoundButton();
Button btn = ButtonFactory.createButton('round');
```

# 简单工厂模式

- 模式结构




```
Product product = Factory.factoryMethod('A');
```

# 简单工厂模式

- 模式结构
  - 简单工厂模式包含如下角色：
    - Factory：工厂角色
    - Product：抽象产品角色
    - ConcreteProduct：具体产品角色

# 简单工厂模式

## 模式分析

**代码复杂，难以维护**

```
public void pay(String type) {
     if(type.equalsIgnoreCase("cash")) {
          //现金支付处理代码
     } else if(type.equalsIgnoreCase("creditcard")) {
          //信用卡支付处理代码
     } else if(type.equalsIgnoreCase("voucher")) {
          //代金券支付处理代码
     } else {
          // 其他代码
     }
}
```

# 简单工厂模式

## 模式分析

```
public abstract class AbstractPay {
    public abstract void pay();
}
```

抽象支付类

```
public class CashPay extends AbstractPay {
    public void pay() {
        //现金支付处理代码
    }
}

public class CreditcardPay extends AbstractPay {
    public void pay() {
        //信用卡支付处理代码
    }
}
```

具体支付类

# 简单工厂模式

## 模式分析

```
public class PayMethodFactory {
    public static AbstractPay getPayMethod(String type) {
        if(type.equalsIgnoreCase("cash")) {
            return new CashPay();          //根据参数创建具体产品
        } else if(type.equalsIgnoreCase("creditcard")) {
            return new CreditcardPay();    //根据参数创建具体产品
        }
    }
}
```

支付工厂

# 简单工厂模式

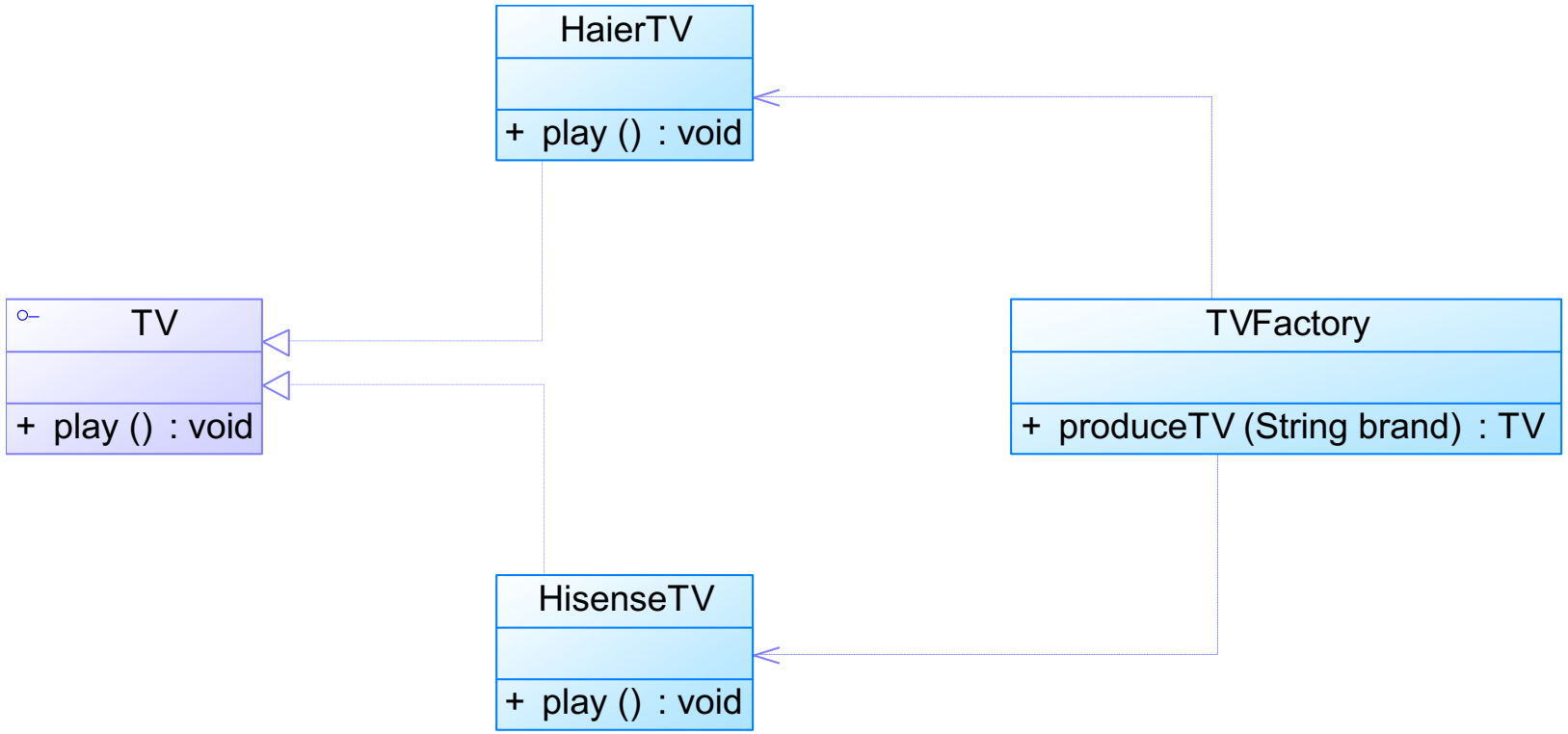
- 模式分析
  - 要点：当需要什么，只需要传入一个正确的参数，就可以获取所需要的对象，而无须知道其创建细节
  - 将对象的创建和业务处理分离可以降低系统的耦合度，使得两者修改起来都相对容易
  - 由于工厂方法是静态方法，使用起来很方便，可通过类名直接调用，只需传入一个简单的参数
  - 最大的问题在于工厂类的职责相对过重，增加新的产品需要修改工厂类的判断逻辑

# 简单工厂模式

- 实例解析
  - 实例一：简单电视机工厂
    - 某电视机厂专为各知名电视机品牌代工生产各类电视机，当需要海尔牌电视机时只需要在调用该工厂的工厂方法时传入参数“Haier”，需要海信电视机时只需要传入参数“Hisense”，工厂可以根据传入的不同参数返回不同品牌的电视机
    - 请使用简单工厂模式来模拟该电视机工厂的生产过程

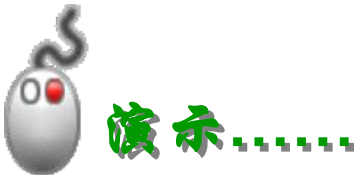
# 简单工厂模式

- 实例解析
  - 实例一：简单电视机工厂



# 简单工厂模式

- 实例解析
  - 实例：简单电视机工厂（tv_factory）



# 简单工厂模式

- 优缺点
  - 优点
    - 工厂类含有必要的判断逻辑，可以决定在什么时候创建哪一个产品类的实例，客户端可以免除直接创建产品对象的责任，而仅仅“消费”产品；简单工厂模式通过这种做法实现了对责任的分割，它提供了专门的工厂类用于创建对象
    - 客户端无须知道所创建的具体产品类的类名，只需要知道具体产品类所对应的参数即可，对于一些复杂的类名，通过简单工厂模式可以减少使用者的记忆量
    - 通过引入配置文件，可以在不修改任何客户端代码的情况下更换和增加新的具体产品类，在一定程度上提高了系统的灵活性

# 简单工厂模式

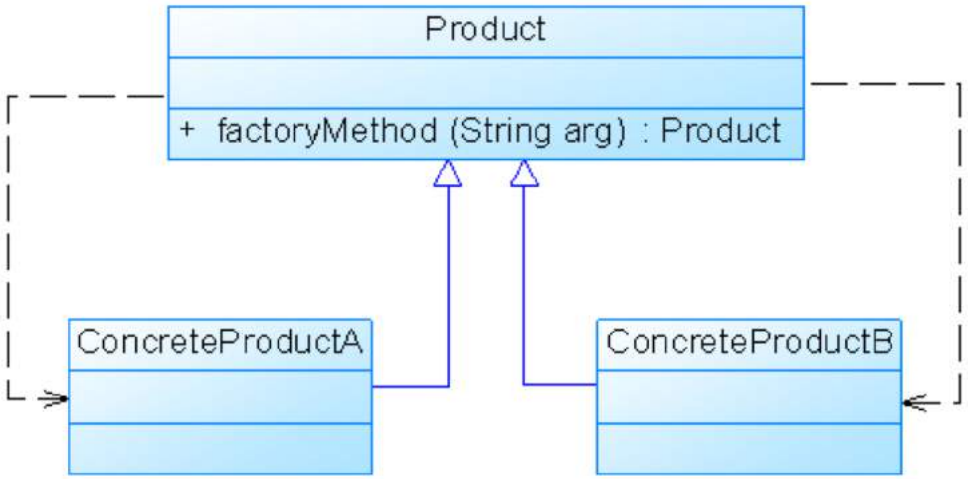
- 优缺点
  - 缺点
    - 由于工厂类集中了所有产品创建逻辑，一旦不能正常工作，整个系统都要受到影响
    - 使用简单工厂模式将会增加系统中类的个数，在一定程序上增加了系统的复杂度和理解难度
    - 系统扩展困难，一旦添加新产品就不得不修改工厂逻辑，在产品类型较多时，有可能造成工厂逻辑过于复杂，不利于系统的扩展和维护
    - 简单工厂模式由于使用了静态工厂方法，造成工厂角色无法形成基于继承的等级结构

# 简单工厂模式

- 适用环境
  - 客户端只知道传入工厂类的参数，对于如何创建对象不关心：客户端既不需要关心创建细节，甚至连类名都不需要记住，只需要知道类型所对应的参数
  - 工厂类负责创建的对象比较少：由于创建的对象较少，不会造成工厂方法中的业务逻辑太过复杂

# 简单工厂模式

- 扩展
  - 简单工厂模式的简化：在有些情况下工厂类可以由抽象产品角色扮演，一个抽象产品类同时也是子类的工厂，也就是说把静态工厂方法写到抽象产品类中




```
Product product = Product.factoryMethod('A');
```

# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）



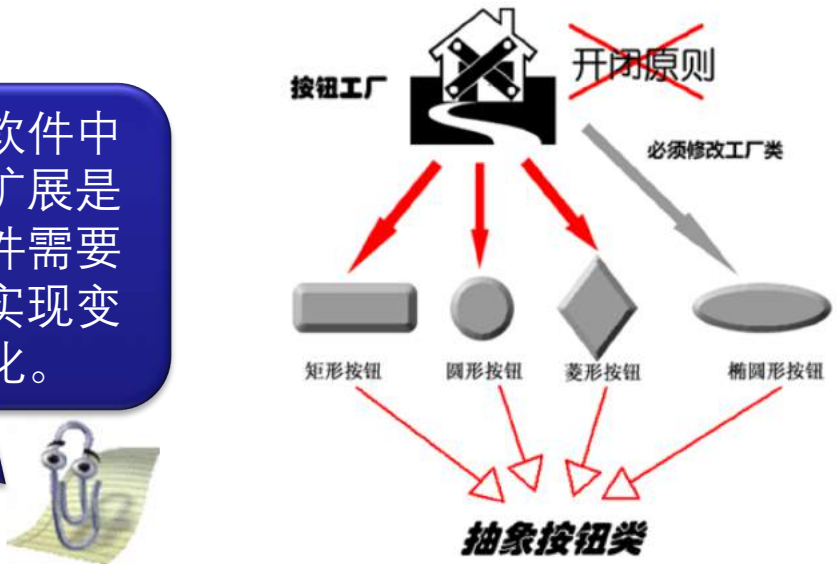
# 工厂方法模式

- 简单工厂模式的不足
  - 在简单工厂模式中，只提供了一个工厂类，该工厂类处于对产品类进行实例化的中心位置，它知道每一个产品对象的创建细节，并决定何时实例化哪一个产品类
  - 简单工厂模式最大的缺点是当有新产品要加入到系统中时，必须修改工厂类，加入必要的处理逻辑。在简单工厂模式中，所有的产品都是由同一个工厂创建，工厂类职责较重，业务逻辑较为复杂，具体产品与工厂类之间的耦合度高，严重影响了系统的灵活性和扩展性
  - 工厂方法模式则可以很好地解决这一问题

# 工厂方法模式

- 模式动机
  - 考虑这样一个系统，按钮工厂类可以返回一个具体的按钮实例，如圆形按钮、矩形按钮、菱形按钮等。在这个系统中，如果需要增加一种新类型的按钮，如椭圆形按钮，那么除了增加一个新的具体产品类之外，还需要修改工厂类的代码，这就使得整个设计在一定程度上违反了“开闭原则”。

在面向对象编程领域中，开闭原则规定“软件中的对象（类，模块，函数等等）应该对于扩展是开放的，但是对于修改是封闭的”。当软件需要变化时，尽量通过扩展软件实体的行为来实现变化，而不是通过修改已有的代码来实现变化。

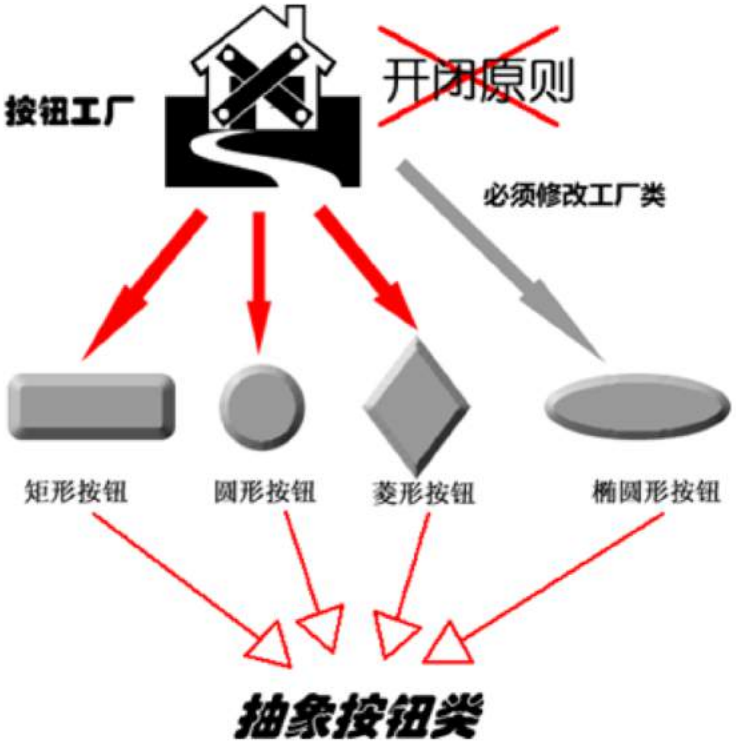


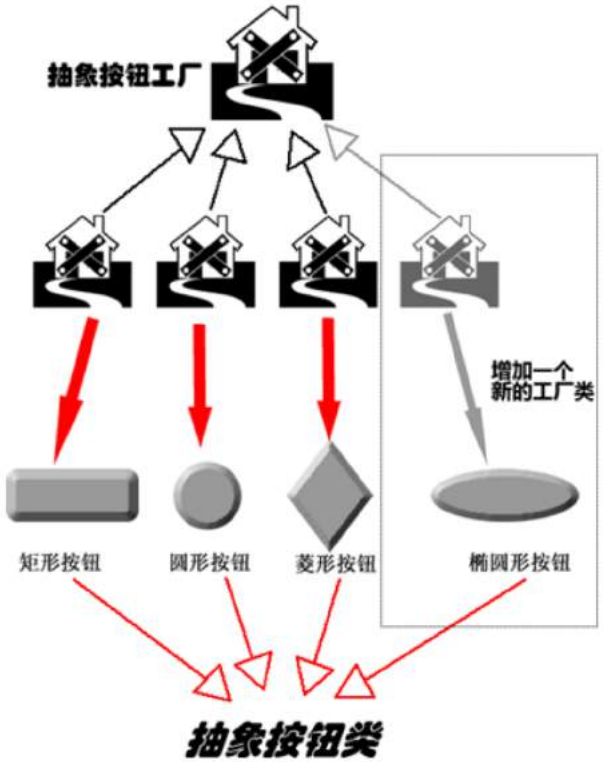
# 工厂方法模式

- 模式动机
  - 现在对该系统进行修改，不再设计一个按钮工厂类来统一负责所有产品的创建，而是将具体按钮的创建过程交给专门的工厂子类去完成
  - 先定义一个抽象的按钮工厂类，再定义具体的按钮工厂类来生成圆形按钮、矩形按钮、菱形按钮等，它们实现在抽象按钮工厂类中定义的方法
  - 这种结构可以在不修改工厂类的情况下引进新的产品：如果出现新的按钮类型，只需要为这种新类型的按钮创建一个具体的工厂类就可以获得该新按钮的实例

# 工厂方法模式

- 模式动机
  - 使用工厂方法模式设计的按钮工厂



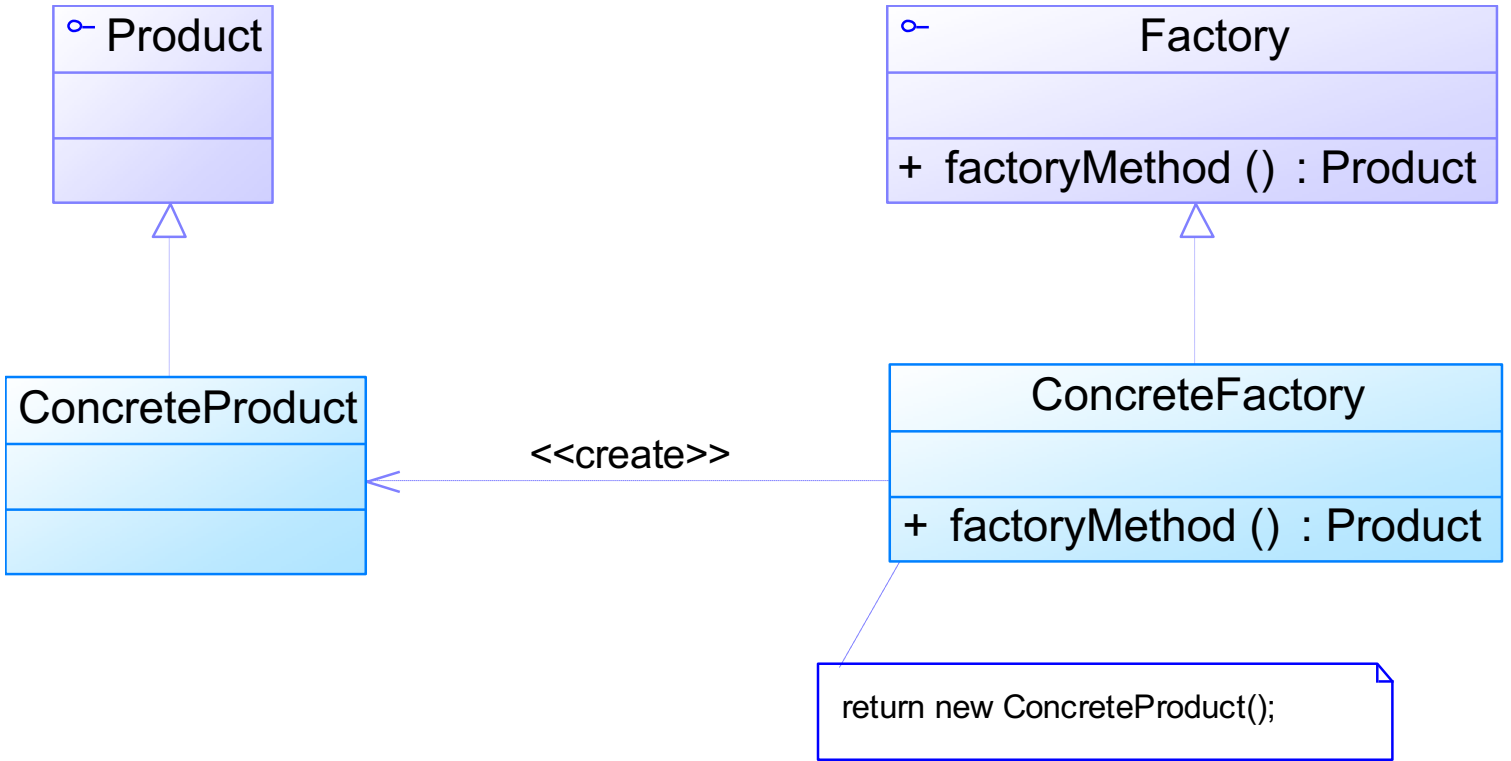
# 工厂方法模式

- 模式定义
  - 工厂方法模式（Factory Method Pattern）也叫虚拟构造器（Virtual Constructor）模式，它属于类创建型模式
  - 在工厂方法模式中，工厂父类负责定义创建产品对象的公共接口，而工厂子类则负责生成具体的产品对象
  - 这样做的目的是将产品类的实例化操作延迟到工厂子类中完成，即通过工厂子类来确定究竟应该实例化哪一个具体产品类

# 工厂方法模式

## 模式结构

工厂方法模式包含如下角色：
- Product: 抽象产品
- ConcreteProduct: 具体产品
- Factory: 抽象工厂
- ConcreteFactory: 具体工厂



# 工厂方法模式

- 模式分析
  - 工厂方法模式是简单工厂模式的进一步抽象和推广。由于使用了面向对象的多态性，工厂方法模式保持了简单工厂模式的优点，而且克服了它的缺点
  - 在工厂方法模式中，核心的工厂类不再负责所有产品的创建，而是将具体创建工作交给子类去做。这个核心类仅仅负责给出具体工厂必须实现的接口，而不负责哪一个产品类被实例化这种细节，这使得工厂方法模式可以允许系统在不修改工厂代码的情况下引进新产品

# 工厂方法模式

- 模式分析

**抽象工厂类**

```
public abstract class PayMethodFactory
{
    public abstract AbstractPay getPayMethod();
}
```

**具体工厂类**

```
public class CashPayFactory extends PayMethodFactory
{
    public AbstractPay getPayMethod()
    {
        return new CashPay();
    }
}
```

# 工厂方法模式

## 模式分析

**客户类代码**

```
PayMethodFactory factory;
AbstractPay payMethod;

factory=new CashPayFactory();
payMethod =factory.getPayMethod();
payMethod.pay();
```

为了提高系统的可扩展性和灵活性，在定义工厂和产品时都必须使用抽象层，如果需要更换产品类，只需要更换对应的工厂即可，其他代码不需要进行任何修改。



# 工厂方法模式

- 模

反射（Reflection）：

在程序运行时获取已知名称的类或已有对象的相关信息的一种机制，包括类的方法、属性、超类等信息，还包括实例的创建和实例类型的判断等。

置文件

```
y</classNa
</
```

反射类名查找

```
//创建一个字符串类型的对象
Class c = Class.forName("String");
Object obj = c.newInstance();
return obj;
```

配置文件代码：

在实际的应用开发中，一般将具体工厂类的实例化过程进行改进，不直接使用new关键字来创建对象，而是将具体类的类名写入配置文件中，再通过Java的反射机制，读取XML格式的配置文件，根据存储在XML文件中的类名字符串生成对象。

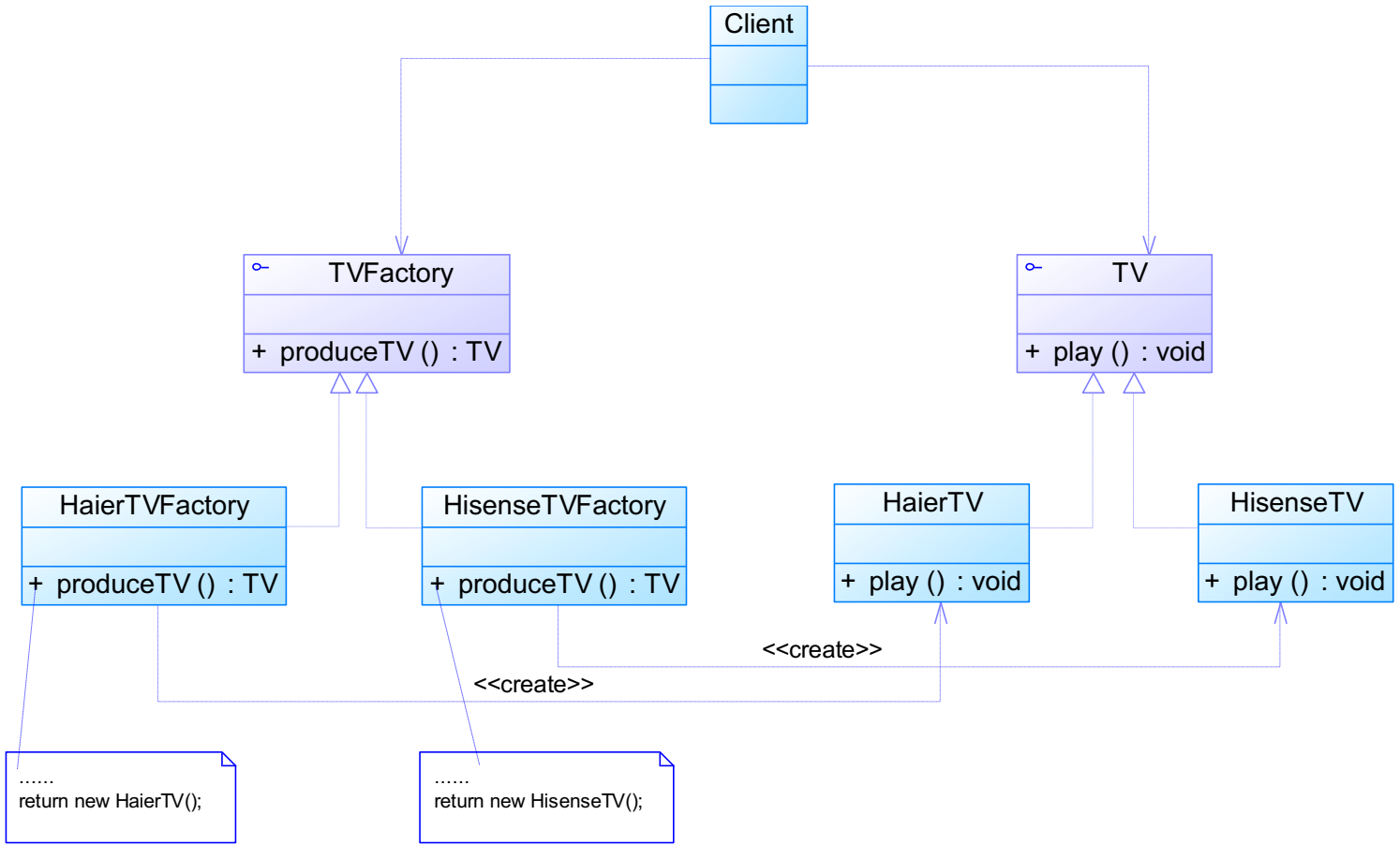


# 工厂方法模式

- 实例解析
  - 实例一：电视机工厂
    - 将原有的工厂进行分割，为每种品牌的电视机提供一个子工厂，海尔工厂专门负责生产海尔电视机，海信工厂专门负责生产海信电视机，如果需要生产TCL电视机或创维电视机，只需要对应增加一个新的TCL工厂或创维工厂即可，原有的工厂无须做任何修改，使得整个系统具有更加的灵活性和可扩展性

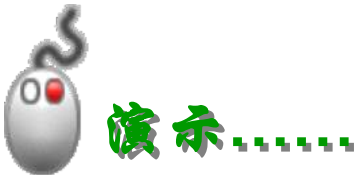
# 工厂方法模式

- 实例解析
  - 实例一：电视机工厂



# 工厂方法模式

- 模式实例与解析
  - 实例：电视机工厂



# 工厂方法模式

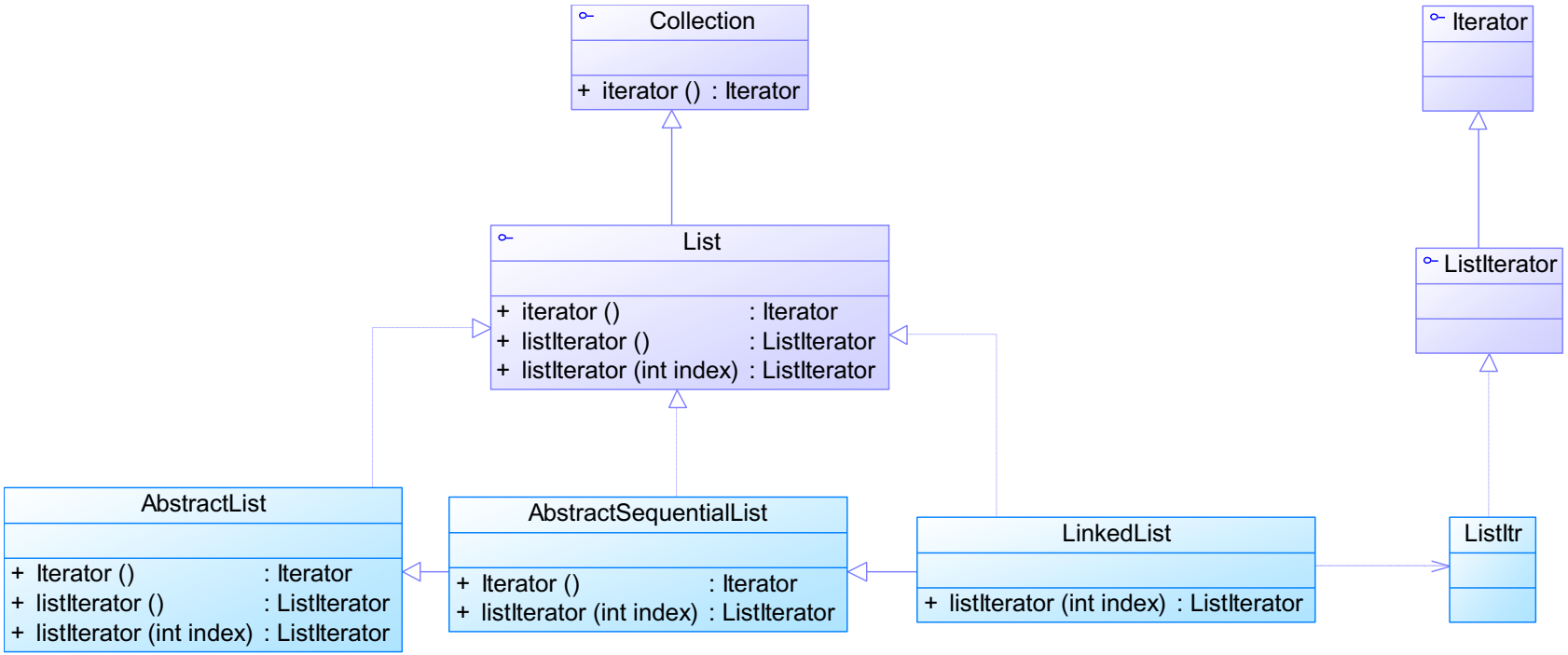
- 模式优缺点
  - 工厂方法模式的优点
    - 在工厂方法模式中，用户只需要关心所需产品对应的工厂，无须关心创建细节，甚至无须知道具体产品类的类名
    - 基于工厂角色和产品角色的多态性是工厂方法模式的关键。它能够使工厂可以自主确定创建何种产品对象，而如何创建这个对象的细节则完全封装在具体工厂内部
    - 在系统中加入新产品时，无须修改抽象工厂和抽象产品提供的接口，无须修改客户端，也无须修改其他的具体工厂和具体产品，而只要添加一个具体工厂和具体产品就可以了。这样，系统的可扩展性也就变得非常好，完全符合“开闭原则”。

# 工厂方法模式

- 模式优缺点
  - 工厂方法模式的缺点
    - 在添加新产品时，需要编写新的具体产品类，而且还要提供与之对应的具体工厂类，系统中类的个数将成对增加，在一定程度上增加了系统的复杂度，有更多的类需要编译和运行，会给系统带来一些额外的开销。
    - 由于考虑到系统的可扩展性，需要引入抽象层，在客户端代码中均使用抽象层进行定义，增加了系统的抽象性和理解难度，且在实现时可能需要用到DOM、反射等技术，增加了系统的实现难度。

# 工厂方法模式

- 模式应用
  - (1) java.util.Collection接口的iterator()方法：



# 工厂方法模式

- 模式应用
  - (2) Java消息服务JMS(Java Messaging Service) ：

```
//使用上下文和 JNDI 得到连接工厂的引用，ctx 是上下文 Context 类型的对象
QueueConnectionFactory qConnFact=(QueueConnectionFactory)ctx.lookup("cfJndi");
//使用连接工厂创建一个连接
QueueConnection qConn=qConnFact.createQueueConnection();
//使用连接创建一个会话
QueueSession qSess=qConn.createQueueSession(false,javax.jms.QueueSession.
AUTO_ACKNOWLEDGE);
//使用上下文和 JNDI 得到消息队列的引用
Queue q=(Queue)ctx.lookup("myQueue");
//使用连接创建一个需要发送的消息类型的实例
QueueSender qSend=qSess.createSender(q);
System.out.println("开始发送消息.....");
```

# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）

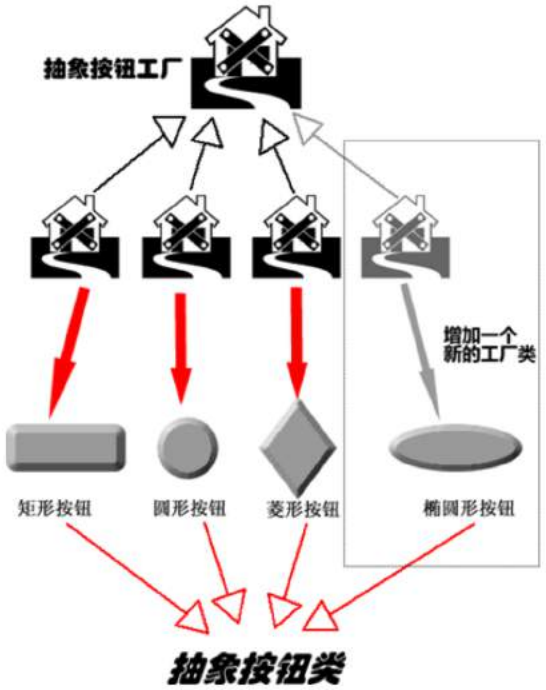


# 第八周分割线

- 周二
- 周四

# 抽象工厂模式

- 模式动机
  - 在工厂方法模式中具体工厂负责生产具体的产品，每一个具体工厂对应一种具体产品
  - 但是有时候我们需要一个工厂可以提供多个产品对象，而不是单一的产品对象

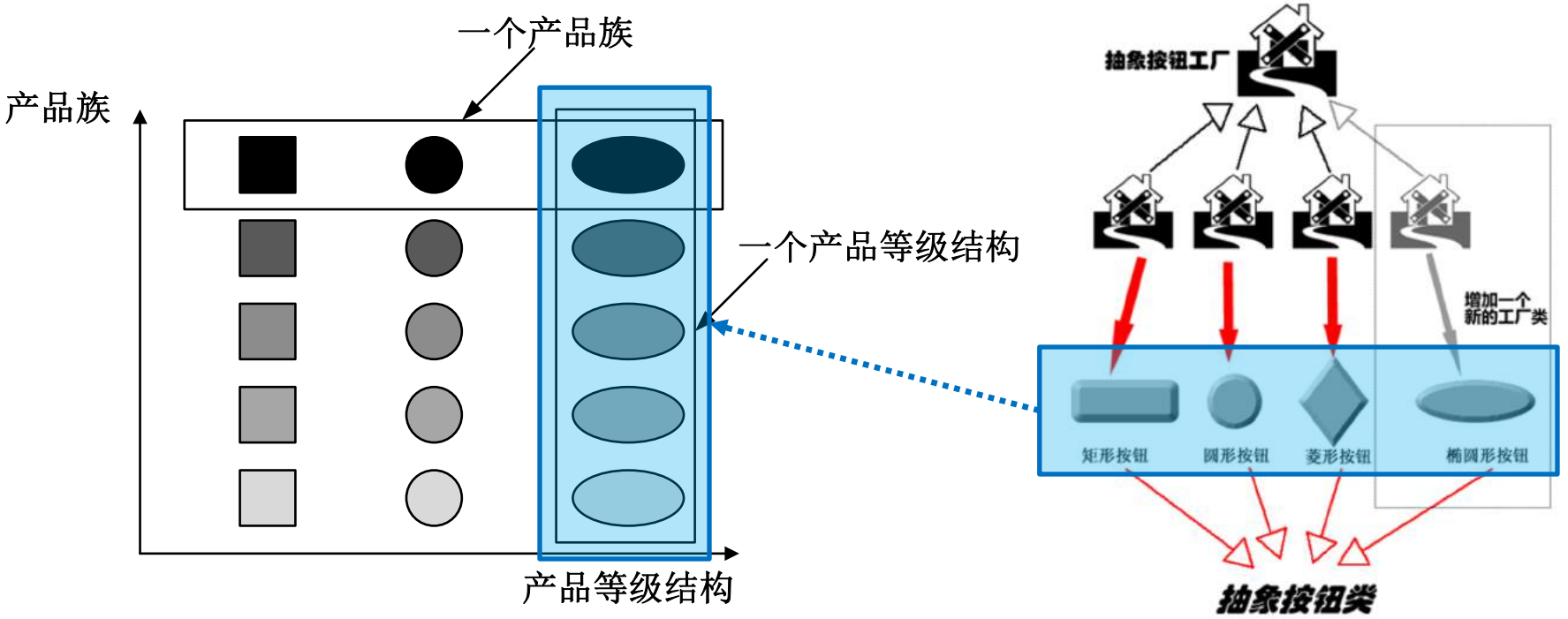


# 抽象工厂模式

- 模式动机
  - 为了更清晰地理解工厂方法模式，需要引入两个概念：
    - 产品等级结构：产品等级结构即产品的继承结构，如一个抽象类是电视机，其子类有海尔电视机、海信电视机、TCL电视机，则抽象电视机与具体品牌的电视机之间构成了一个产品等级结构，抽象电视机是父类，而具体品牌的电视机是其子类。
    - 产品族：在抽象工厂模式中，产品族是指由同一个工厂生产的，位于不同产品等级结构中的一组产品，如海尔电器工厂生产的海尔电视机、海尔电冰箱，海尔电视机位于电视机产品等级结构中，海尔电冰箱位于电冰箱产品等级结构中。

# 抽象工厂模式

- 模式动机
  - 产品族与产品等级结构示意图

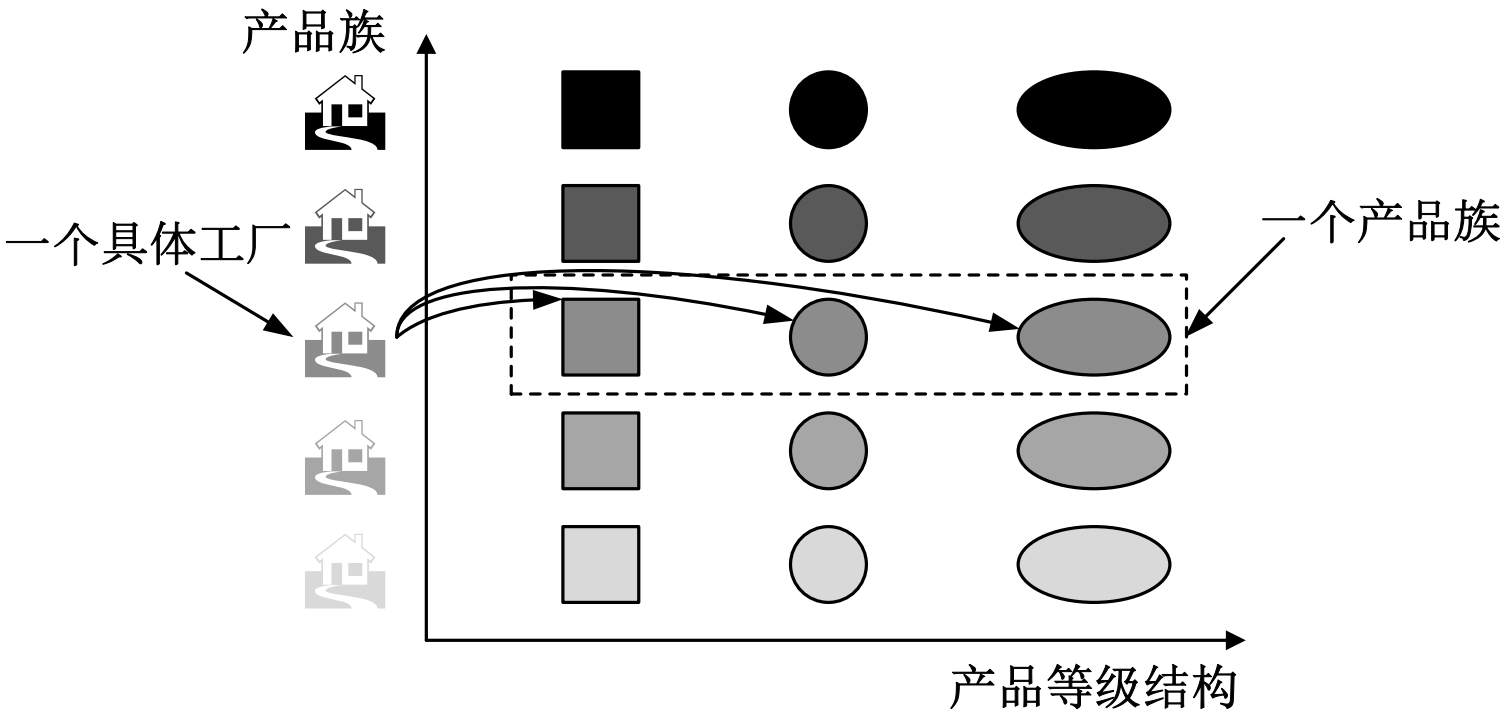


# 抽象工厂模式

- 模式动机
  - 当所需生产的具体产品不是一个简单的对象，而是多个位于不同产品等级结构中属于不同产品族的具体产品时需要使用抽象工厂模式
  - 抽象工厂模式是所有形式的工厂模式中最为抽象和最具一般性的一种形态
  - 抽象工厂模式与工厂方法模式最大的区别在于，工厂方法模式针对的是一个产品等级结构（电视机），而抽象工厂模式则需要面对多个产品等级结构（电视机、洗衣机、电冰箱），即一系列产品族

# 抽象工厂模式

- 模式动机
  - 抽象工厂模式示意图



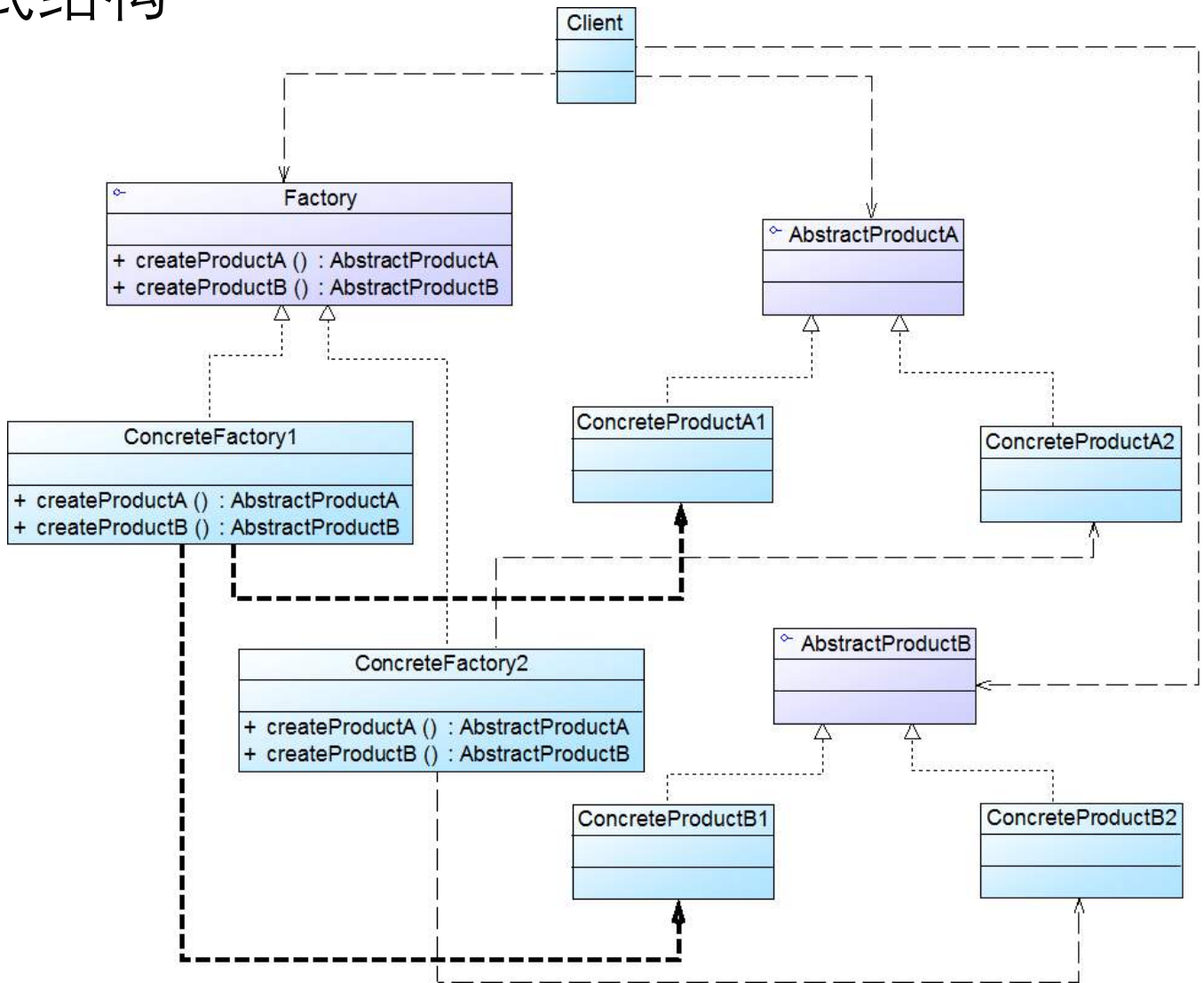
# 抽象工厂模式

- 模式定义
  - 抽象工厂模式（Abstract Factory Pattern）提供一个创建一系列相关或相互依赖对象的接口，而无须指定它们具体的类
  - 抽象工厂模式属于创建型模式

# 抽象工厂模式

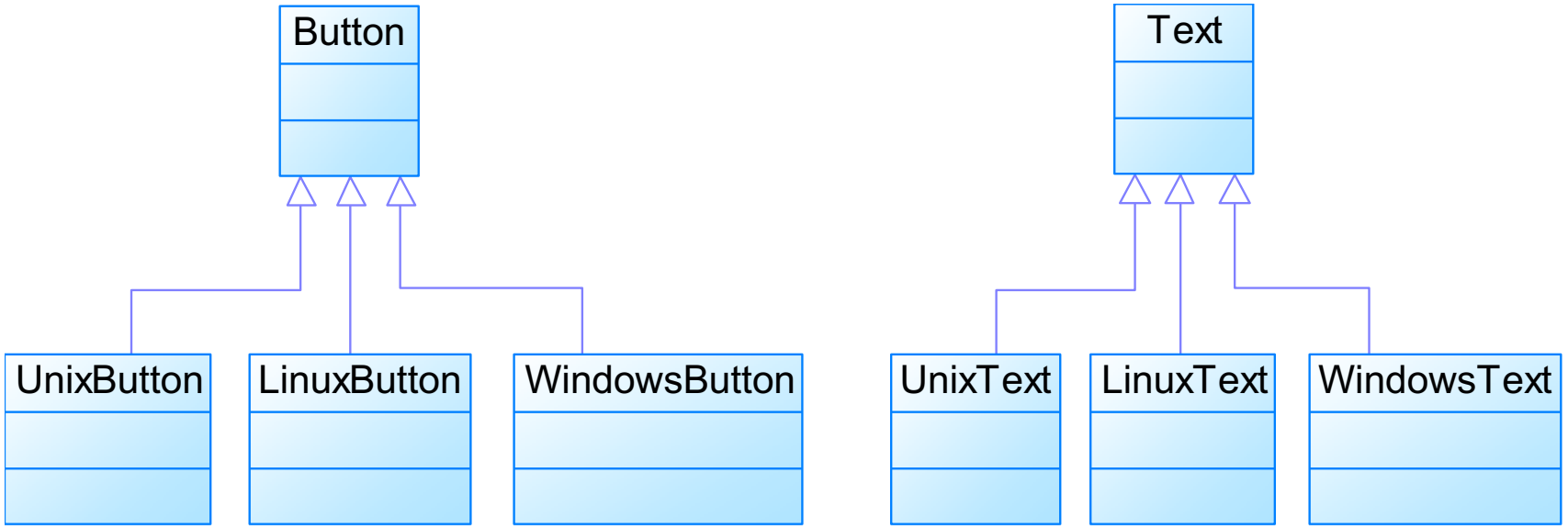
工厂方法模式包含如下角色：
- Product: 抽象产品
- ConcreteProduct: 具体产品
- Factory: 抽象工厂
- ConcreteFactory: 具体工厂

## 模式结构



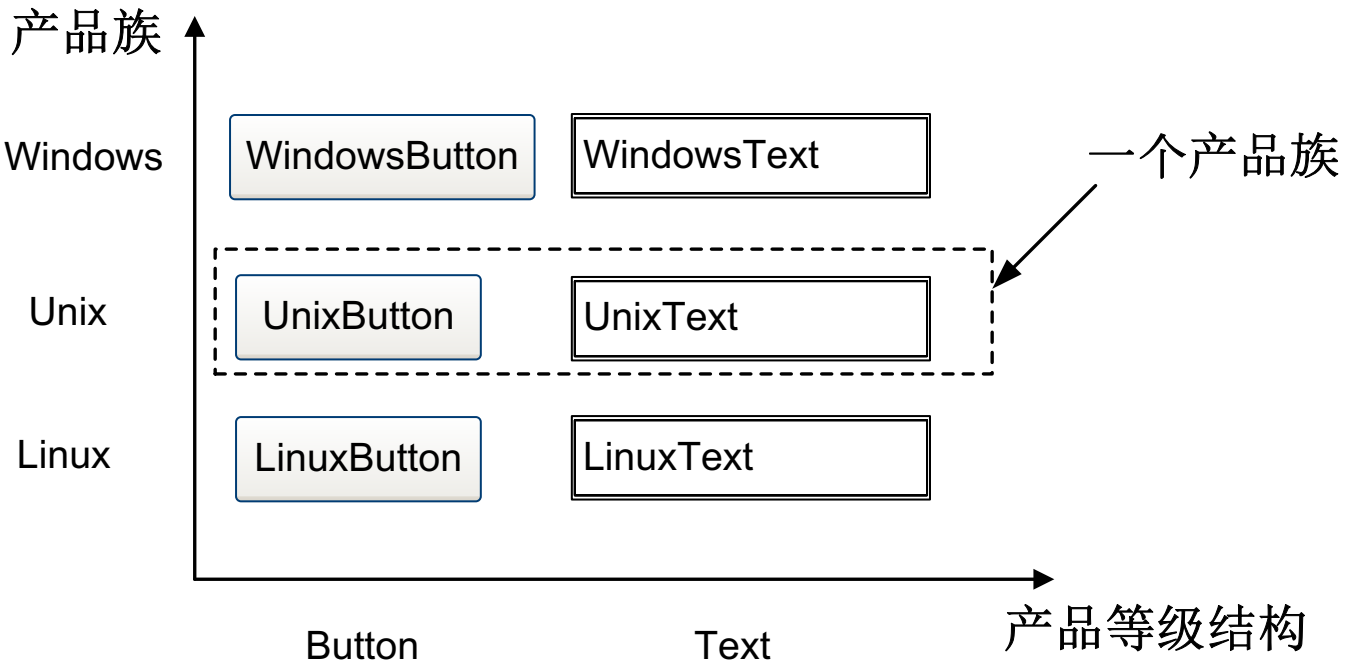
# 抽象工厂模式

- 模式分析


Button
UnixButton
LinuxButton
WindowsButton
Text
UnixText
LinuxText
WindowsText

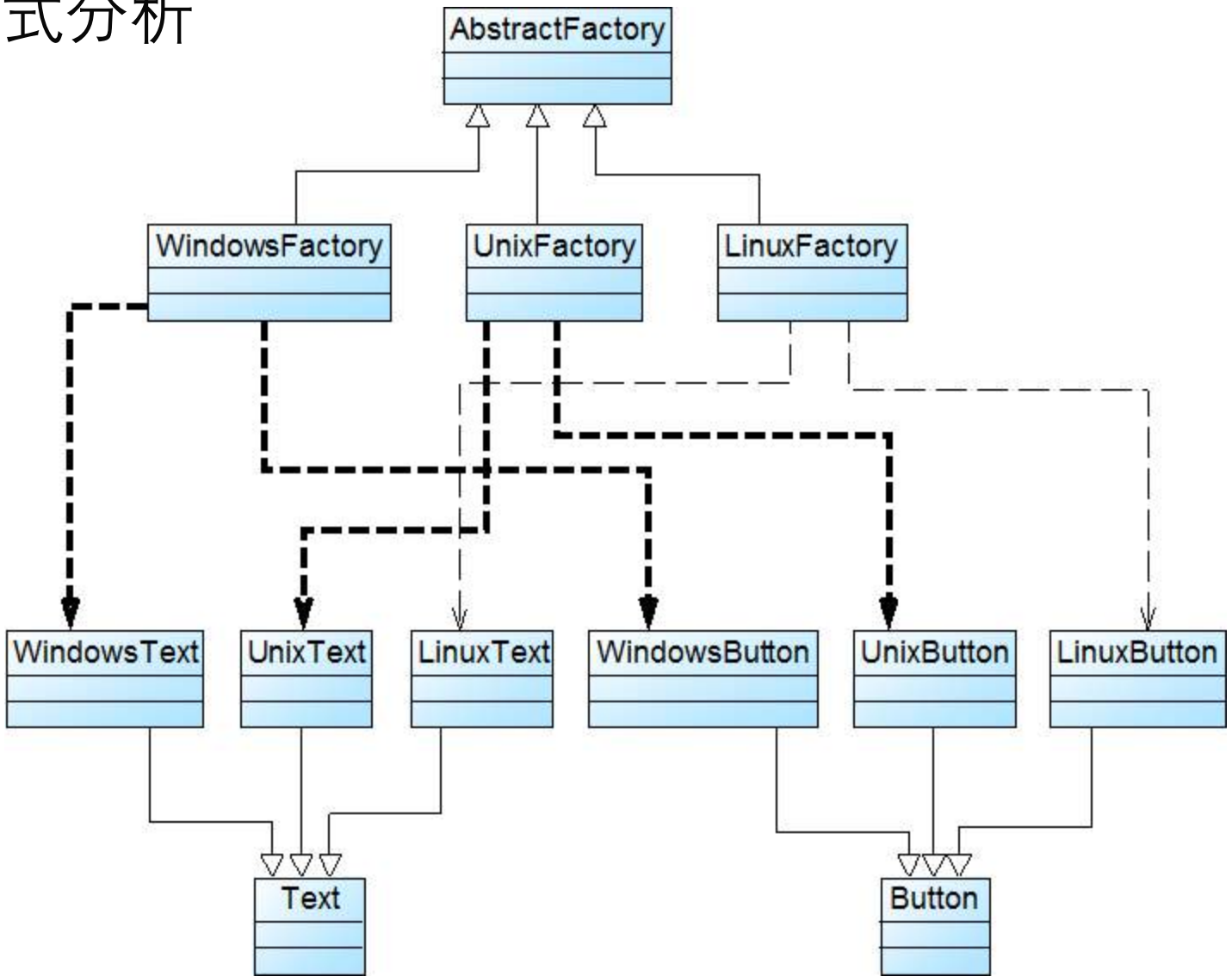
# 抽象工厂模式

- 模式分析


产品族
Windows
WindowsButton
WindowsText
一个产品族
Unix
UnixButton
UnixText
Linux
LinuxButton
LinuxText
Button
Text
产品等级结构

# 抽象工厂模式

- 模式分析


AbstractFactory
WindowsFactory
UnixFactory
LinuxFactory
WindowsText
UnixText
LinuxText
WindowsButton
UnixButton
LinuxButton
Text
Button

# 抽象工厂模式




## 模式分析

```
public abstract class Factory {
    public abstract Button createButton();
    public abstract Text createText();
}
```

```
public class LinuxFactory extends Factory {
    public LinuxButton createButton() {
        return new LinuxButton();
    }
    public LinuxText createText() {
        return new LinuxText();
    }
}
```

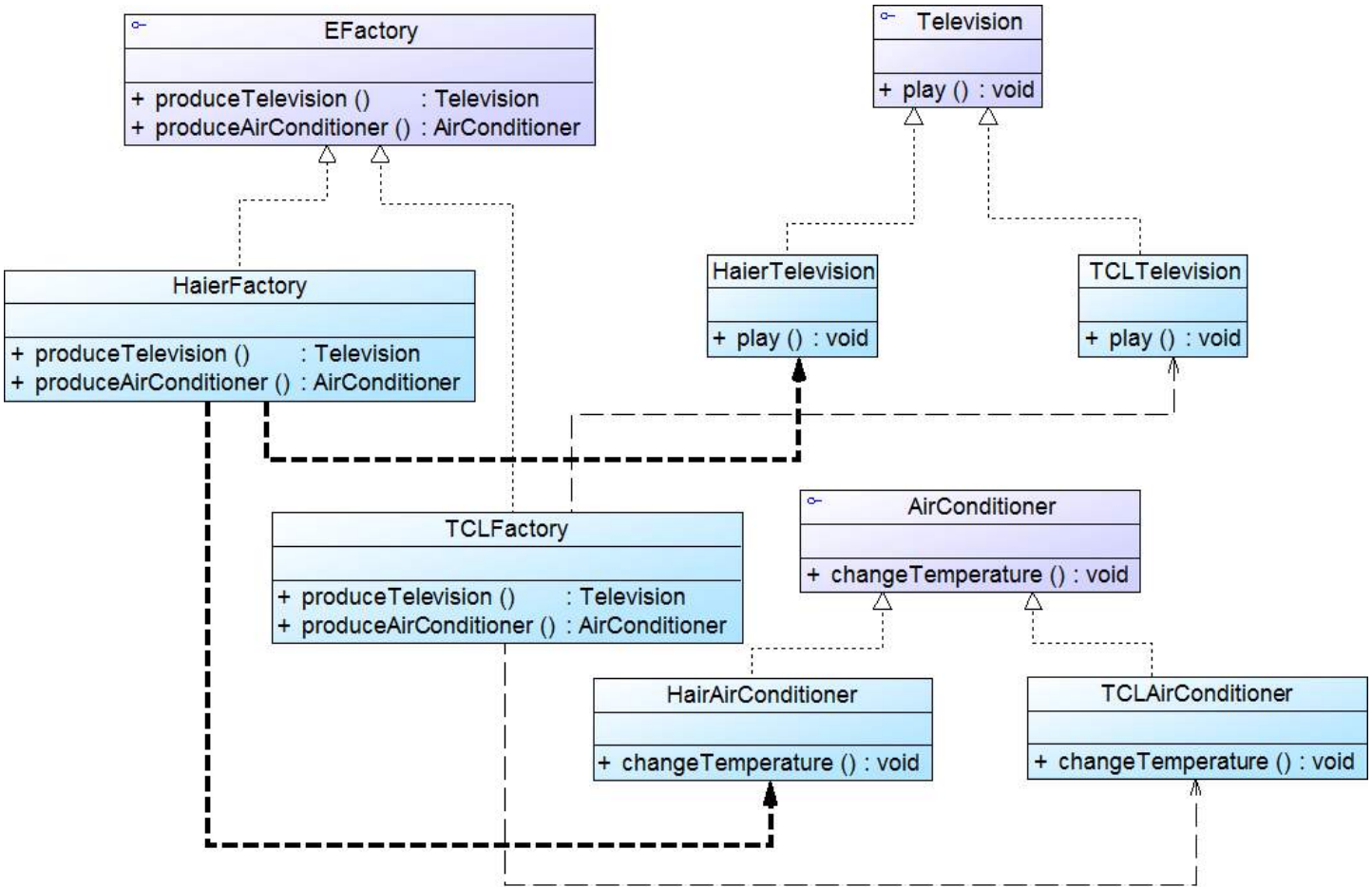
```
Factory factory = (Factory) Class.forName("LinuxFactory");
factory.createButton();
factory.createText();
```

# 抽象工厂模式

- 模式实例与解析
  - 实例一：电器工厂
    - 一个电器工厂可以产生多种类型的电器，如海尔工厂可以生产海尔电视机、海尔空调等，TCL工厂可以生产TCL电视机、TCL空调等，相同品牌的电器构成一个产品族，而相同类型的电器构成了一个产品等级结构
    - 请使用抽象工厂模式模拟该场景

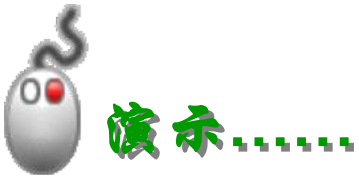
# 抽象工厂模式

- 模式实例与解析
  - 实例一：电器工厂



# 抽象工厂模式

- 模式实例与解析
  - 实例：电器工厂
    - 参考代码 (abstract_factory)



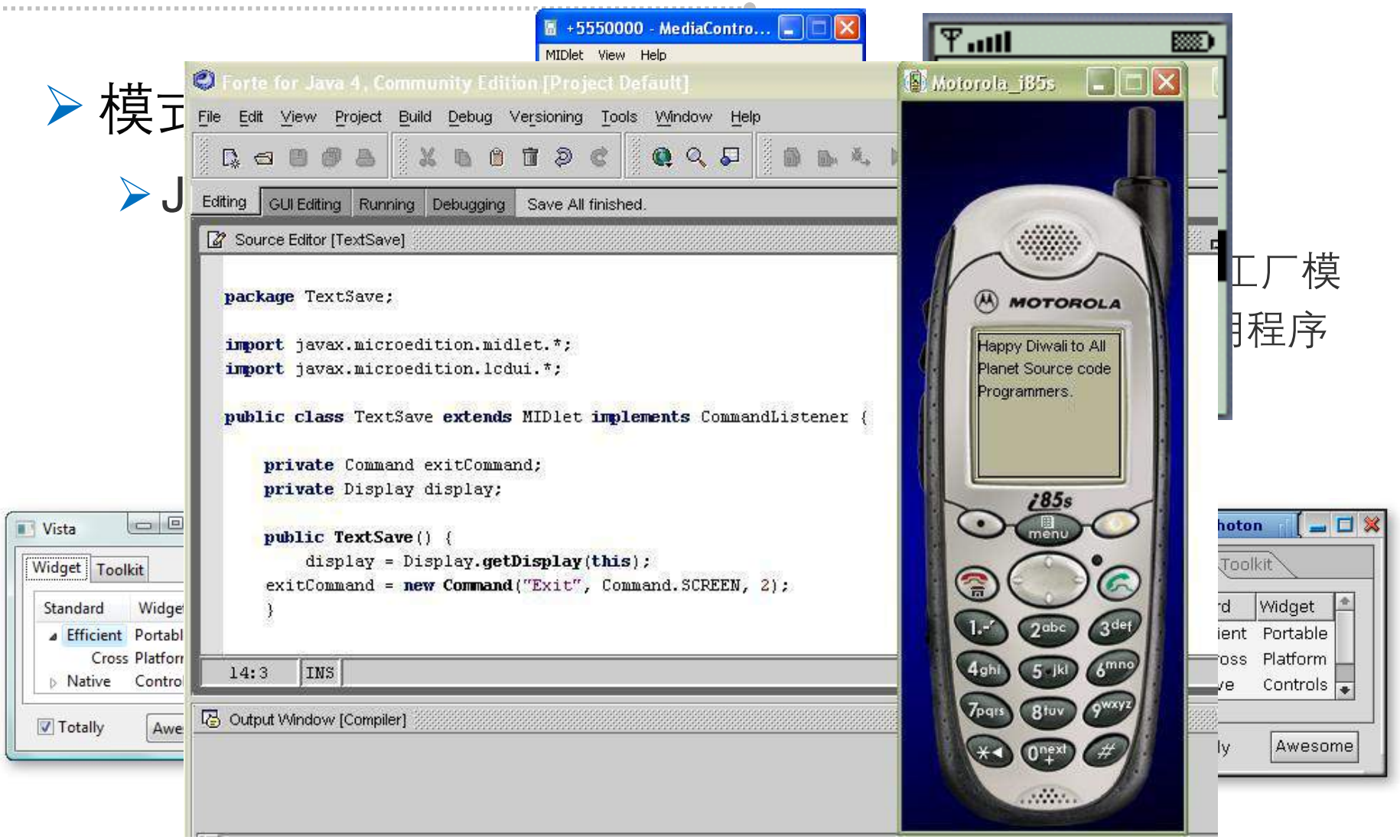
# 抽象工厂模式

- 模式优缺点
  - 抽象工厂模式的优点
    - 抽象工厂模式隔离了具体类的生成，使得客户并不需要知道什么被创建。由于这种隔离，更换一个具体工厂就变得相对容易
    - 当一个产品族中的多个对象被设计成一起工作时，它能够保证客户端始终只使用同一个产品族中的对象。这对一些需要根据当前环境来决定其行为的软件系统来说，是一种非常实用的设计模式（跨平台、换皮肤）
    - 增加新的具体工厂和产品族很方便，无须修改已有系统，符合“开闭原则”

# 抽象工厂模式

- 模式适用环境
  - 在以下情况下可以使用抽象工厂模式：
    - 一个系统不应当依赖于产品类实例如何被创建、组合和表达的细节，这对于所有类型的工厂模式都是重要的
    - 系统中有多于一个的产品族，而每次只使用其中某一产品族
    - 属于同一个产品族的产品将在一起使用，这一约束必须在系统的设计中体现出来
    - 系统提供一个产品类的库，所有的产品以同样的接口出现，从而使客户端不依赖于具体实现

# 抽象工厂模式




[软件体系结构考古团：Java Mobile (J2ME) 时代的手机应用]

# 抽象工厂模式

- 模式应用
  - 在很多软件系统中，要求界面中的按钮、菜单、窗口等元素具有一致的风格，例如，同时具有“Windows风格”或者“Unix风格”的元素，为了实现这种要求，可以使用抽象工厂模式。









# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）



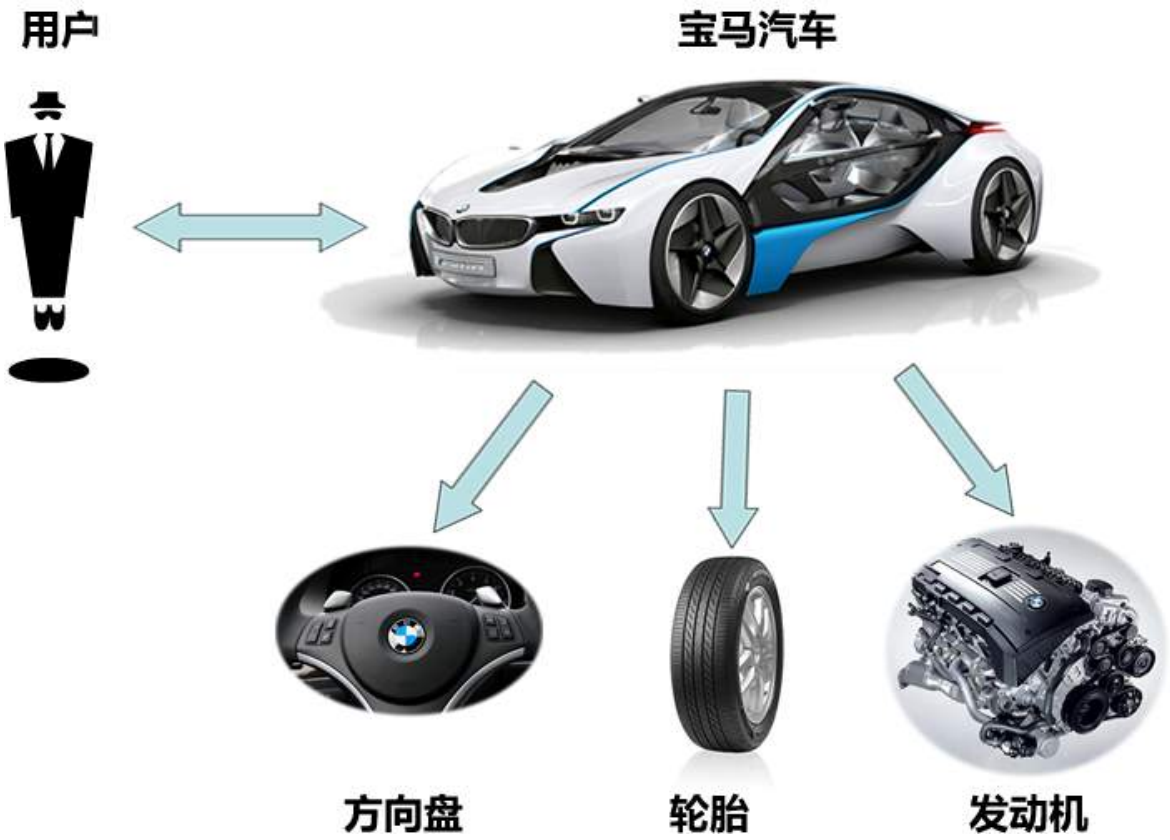
# 建造者模式

## 模式动机

- 无论是在现实世界中还是在软件系统中，都存在一些复杂的对象，它们拥有多个组成部分，如汽车，它包括车轮、方向盘、发送机等各种部件。而对于大多数用户而言，无须知道这些部件的装配细节，也几乎不会使用单独某个部件，而是使用一辆完整的汽车，可以通过建造者模式对其进行设计与描述，建造者模式可以将部件和其组装过程分开，一步一步创建一个复杂的对象。用户只需要指定复杂对象的类型就可以得到该对象，而无须知道其内部的具体构造细节。

# 建造者模式

## 模式动机



# 建造者模式

## 模式动机

- 在软件开发中，也存在大量类似汽车一样的复杂对象，它们拥有一系列成员属性，这些成员属性中有些是引用类型的成员对象。而且在这些复杂对象中，还可能存在一些限制条件，如某些属性没有赋值则复杂对象不能作为一个完整的产品使用；有些属性的赋值必须按照某个顺序，一个属性没有赋值之前，另一个属性可能无法赋值等
- 复杂对象相当于一辆有待建造的汽车，而对象的属性相当于汽车的部件，建造产品的过程就相当于组合部件的过程。由于组合部件的过程很复杂，因此，这些部件的组合过程往往被“外部化”到一个称作建造者的对象里，建造者返还给客户端的是一个已经建造完毕的完整产品对象，而用户无须关心该对象所包含的属性以及它们的组装方式

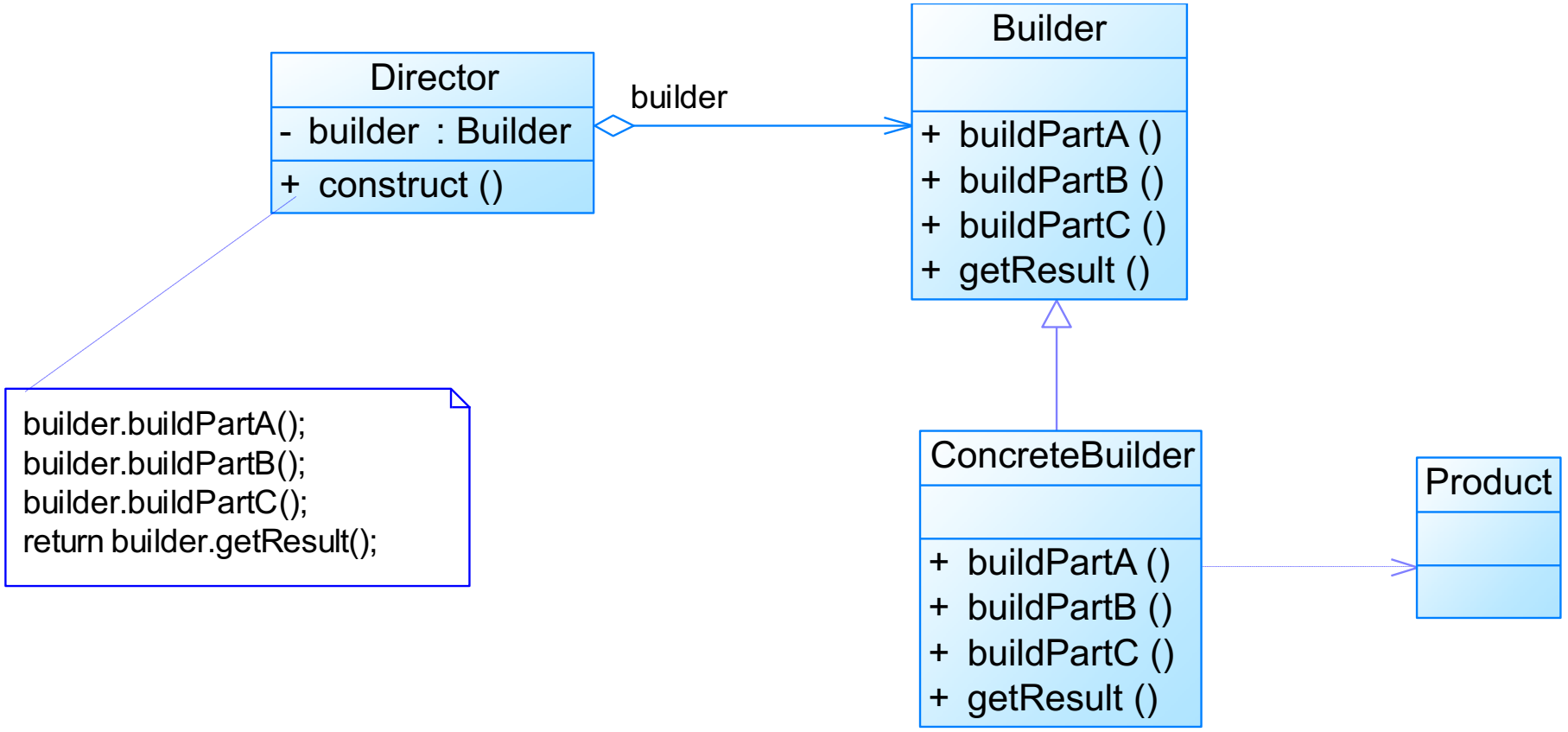
# 建造者模式

- 模式定义
  - 建造者模式（Builder Pattern）将一个复杂对象的构建与它的表示分离，使得同样的构建过程可以创建不同的表示
  - 建造者模式是一步一步创建一个复杂的对象，它允许用户只通过指定复杂对象的类型和内容就可以构建它们，用户不需要知道内部的具体构建细节。建造者模式属于对象创建型模式。建造者模式又可以称为生成器模式

# 建造者模式

建造者模式包含如下角色：
- Builder：抽象建造者
- ConcreteBuilder：具体建造者
- Director：指挥者
- Product：产品角色

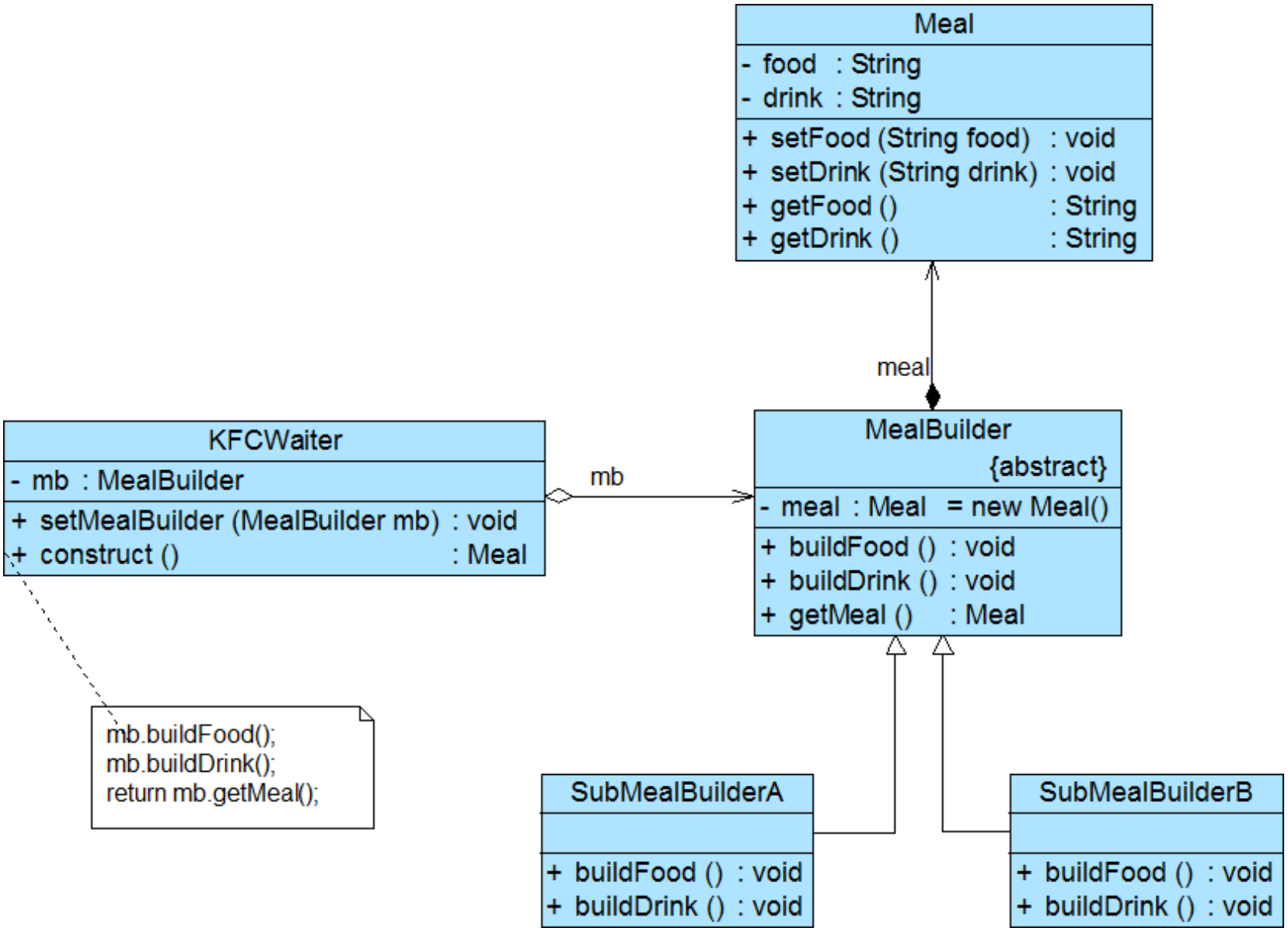
## 模式结构



# 建造者模式

- 模式实例解析
  - 实例：KFC套餐
    - 建造者模式可以用于描述KFC如何创建套餐：套餐是一个复杂对象，它一般包含主食（如汉堡、鸡肉卷等）和饮料（如果汁、可乐等）等组成部分，不同的套餐有不同的组成部分，而KFC的服务员（Director）可以根据顾客的要求，一步一步装配这些组成部分，构造一份完整的套餐，然后返回给顾客

# 建造者模式


Meal
- food : String
- drink : String
+ setFood (String food) : void
+ setDrink (String drink) : void
+ getFood () : String
+ getDrink () : String
meal
MealBuilder
{abstract}
- meal : Meal = new Meal()
+ buildFood () : void
+ buildDrink () : void
+ getMeal () : Meal
KFCWaiter
- mb : MealBuilder
+ setMealBuilder (MealBuilder mb) : void
+ construct () : Meal
mb
mb.buildFood();
mb.buildDrink();
return mb.getMeal();
SubMealBuilderA
+ buildFood () : void
+ buildDrink () : void
SubMealBuilderB
+ buildFood () : void
+ buildDrink () : void

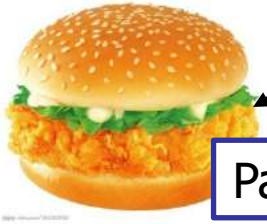
# 建造者模式




**KFC使用了建造者模式**

- 建造者模式实例与解析





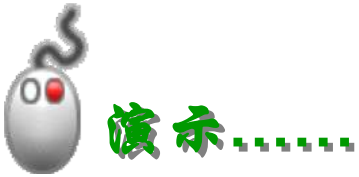







# 建造者模式

- 建造者模式实例与解析
  - 实例：KFC套餐
    - 参考代码 (builder)



# 建造者模式

- 模式分析
  - 客户端类代码片段

```
Builder builder = new ConcreteBuilder();
Director director = new Director(builder);
Product product = director.construct();
```

  - 在客户端代码中，无须关心产品对象的具体组装过程，只需确定具体建造者的类型即可，建造者模式将复杂对象的构建与对象的表现分离开来，这样使得同样的构建过程可以创建出不同的表现

# 建造者模式

- 模式分析
  - 一个典型的复杂对象其类代码示例如下

```
public class Product
{
    private String partA; //可以是任意类型
    private String partB;
    private String partC;
    //partA 的 Getter 方法和 Setter 方法省略
    //partB 的 Getter 方法和 Setter 方法省略
    //partC 的 Getter 方法和 Setter 方法省略
}
```

# 建造者模式

- 模式分析
  - 抽象建造者类中定义了产品的创建方法和返回方法，典型代码如下

```
public abstract class Builder
{
    protected Product product=new Product();

    public abstract void buildPartA();
    public abstract void buildPartB();
    public abstract void buildPartC();

    public Product getResult()
    {
        return product;
    }
}
```

# 建造者模式

- 模式分析
  - 建造者模式的结构中还引入了一个指挥者类Director，该类的作用主要有两个：一方面它隔离了客户与生产过程；另一方面它负责控制产品的生成过程
  - 指挥者针对抽象建造者编程，客户端只需要知道具体建造者的类型，即可通过指挥者类调用建造者的相关方法，返回一个完整的产品对象

# 建造者模式

- 模式分析
  - 指挥者类的代码示例如下

```
public class Director
{
     private Builder builder;
     public Director(Builder builder) {
          this.builder=builder;
     }
     public void setBuilder(Builder builder) {
          this.builder=builer;
     }
     public Product construct() {
          builder.buildPartA();
          builder.buildPartB();
          builder.buildPartC();
          return builder.getResult();
     }
}
```

# 建造者模式

## 模式优缺点

### 建造者模式的优点

- 客户端不必知道产品内部组成的细节，将产品本身与产品的创建过程解耦，使得相同的创建过程可以创建不同的产品对象
- 每一个具体建造者都相对独立，可以很方便地替换建造者或增加新的建造者，用户使用不同的具体建造者即可得到不同的产品对象
- 将复杂产品的创建步骤分解在不同的方法中，使得创建过程更加清晰，可以更加精细地控制产品的创建过程
- 增加新的具体建造者无须修改原有类库的代码，指挥者类针对抽象建造者类编程，系统扩展方便，符合“开闭原则”

# 建造者模式

- 模


茶港 港式茶餐厅
HONG KONG RESTAURANT
快生活|慢味蕾 全国连锁品牌
港式烧味
港式小点
港式肠粉
港式主食
港式主食
精品粤菜
明炉烤鸭 39元/例 58元/半只
广式水晶虾饺 18元/份
鲜虾肠粉 16元/份
星洲炒米粉 29元/份
烧鸭饭 29元/份
火鸭丝蚊米粉 26元/份
叉烧包 12元/份
叉烧脆瓜肠粉 15元/份
干炒牛河 29元/份
叉烧饭 30元/份
榄菜肉沫蒸娃娃菜 12元/份
烧味双拼 38元/份
流沙包 12元/份
鲜虾烧麦 16元/份
瑶柱海鲜炒饭 32元/份
排骨煲仔饭 30元/份
生炒水东芥 28元/份
旺角蜜汁叉烧 42元/份
豉汁蒸排骨 18元/份
海鲜蒸蛋 15元/份
宫爆鸡丁饭 23元/份
牛肉饭 26元/份
豆豉鲮鱼油麦菜 26元/份
手撕盐焗鸡 38元/只
豉汁蒸凤爪 16元/份
蒜茸蒸开边虾 18元/份
皮蛋瘦肉粥 16元/份
南瓜山药粥 15元/份
每日例汤 10元/例
港式菠萝油 7元/个



# 建造者模式

- 模式应用
  - JavaMail（一步一步构造一个完整的邮件对象，然后发送）

```
//由邮件会话对象新建一个邮件消息对象
MimeMessage message = new MimeMessage(session);
//设置邮件地址
InternetAddress from = new InternetAddress("alice@mail.com");
message.setFrom(from);//设置发件人
InternetAddress to = new InternetAddress(to_mail);
message.setRecipient(Message.RecipientType.TO, to); //设置收件人
message.setSubject(to_title);//设置主题
message.setText(to_content);//设置信件内容
message.setSentDate(new Date());//设置发信时间
message.saveChanges();//存储邮件信息
Transport transport = session.getTransport("smtp");
transport.connect("smtp.test.com", "test", "test");
transport.sendMessage(message, message.getAllRecipients());
```

# 建造者模式

- 模式应用
  - 在很多游戏软件中，地图包括天空、地面、背景等组成部分，人物角色包括人体、服装、装备等组成部分，可以使用建造者模式对其进行设计，通过不同的具体建造者创建不同类型的地图或人物



Game Builder @ STEAM

Minecraft: Working Cell Phone



# 建造者模式

- 模式模式







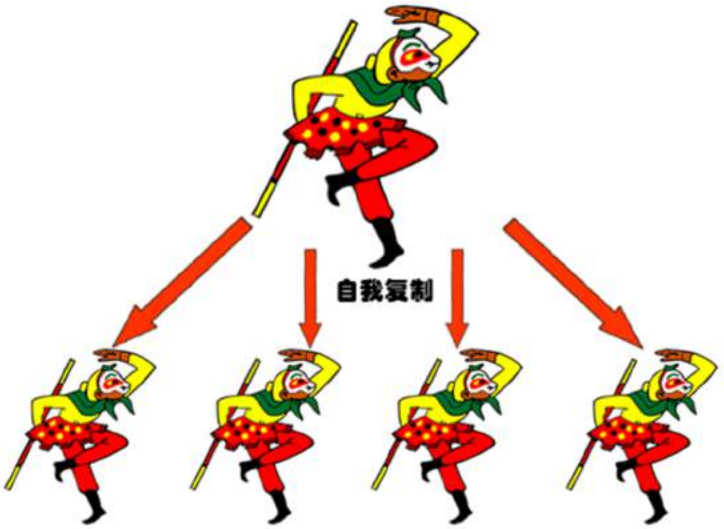


造者

- 如果 ，生产一个产品族的产品 厂，通过对部件的组装可

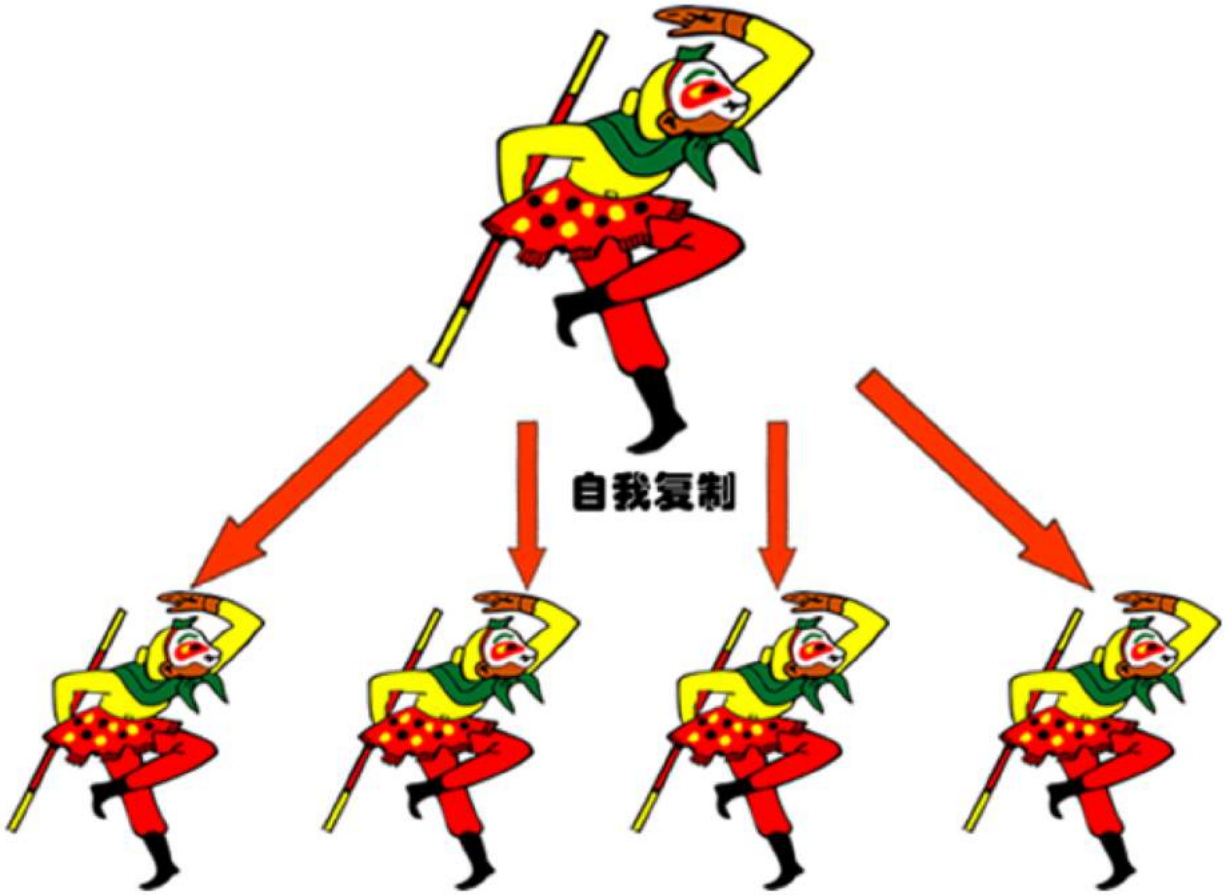
# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）



# 原型模式

- 模式动机

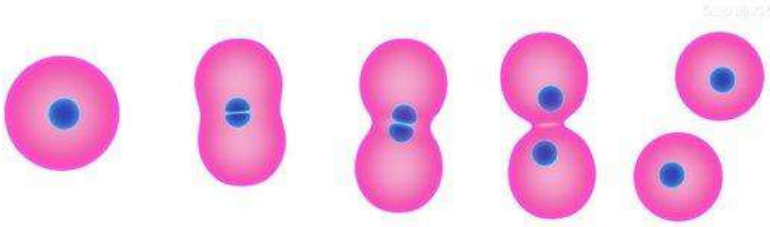


# 原型模式

- 模式动机
  - 在面向对象系统中，使用原型模式来复制一个对象自身，从而克隆出多个与原型对象一模一样的对象
  - 在软件系统中，有些对象的创建过程较为复杂，而且有时候需要频繁创建，原型模式通过给出一个原型对象来指明所要创建的对象的类型，然后用复制这个原型对象的办法创建出更多同类型的对象

# 原型模式

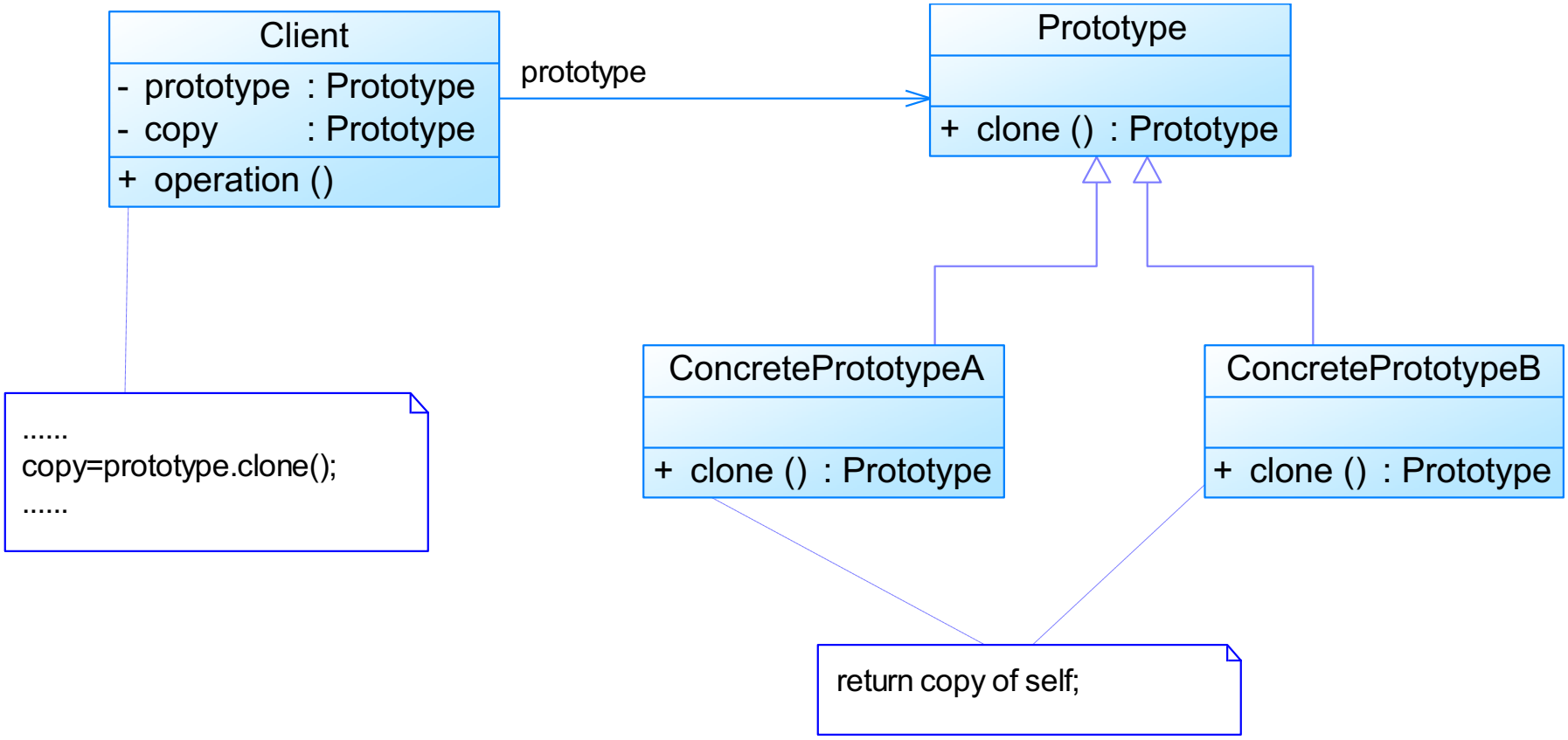
- 模式定义
  - 原型模式（Prototype Pattern）是一种创建型模式，用原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象
  - 原型模式允许一个对象再创建另外一个可定制的对象，无须知道任何创建的细节
  - 原型模式的基本工作原理是通过将一个原型对象传给那个要发动创建的对象，这个要发动创建的对象通过请求原型对象拷贝原型自己来实现创建过程



# 原型模式

原型模式包含如下角色
- Prototype：抽象原型类
- ConcretePrototype：具体原型类
- Client：客户类

## 模式结构



# 原型模式

## 模式分析

- 在原型模式结构中定义了一个抽象原型类，所有的Java类都继承自java.lang.Object，而Object类提供一个clone()方法，可以将一个Java对象复制一份。因此在Java中可以直接使用Object提供的clone()方法来实现对象的克隆，Java语言中的原型模式实现很简单
- 能够实现克隆的Java类必须实现一个标识接口Cloneable，表示这个Java类支持复制。如果一个类没有实现这个接口但是调用了clone()方法，Java编译器将抛出一个CloneNotSupportedException异常

# 原型模式

- 模式分析
  - 示例代码

```
public class PrototypeDemo implements Cloneable {
    public Object clone() {
        Object object = null;
        try {
            object = super.clone();

        } catch (CloneNotSupportedException exception) {
            System.err.println("Not support cloneable");

        }
        return object;
    }
}
```

# 原型模式

- 模式分析
  - Java语言提供的clone()方法将对象复制了一份并返回给调用者。一般而言，clone()方法满足：
    - 对任何的对象x，都有x.clone() !=x，即克隆对象与原对象不是同一个对象
    - 对任何的对象x，都有x.clone().getClass()==x.getClass()，即克隆对象与原对象的类型一样

# 原型模式

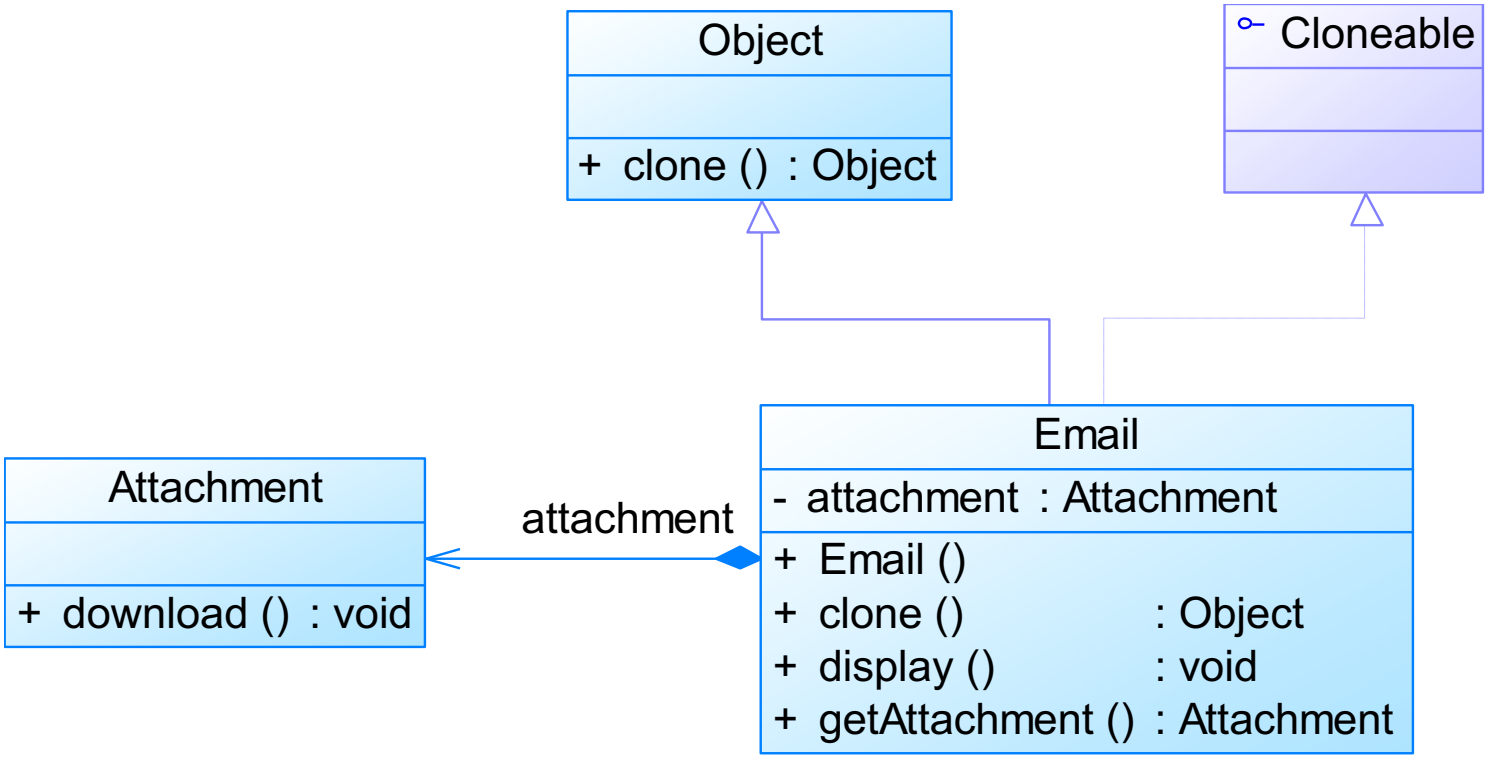
## 原型模式实例与解析

### 实例一：邮件复制

- 由于邮件对象包含的内容较多（如发送者、接收者、标题、内容、日期、附件等），某系统中现需要提供一个邮件复制功能，对于已经创建好的邮件对象，可以通过复制的方式创建一个新的邮件对象，如果需要改变某部分内容，无须修改原始的邮件对象，只需要修改复制后得到的邮件对象即可。使用原型模式设计该系统

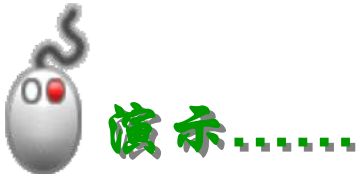
# 原型模式

- 原型模式实例与解析
  - 实例一：邮件复制



# 原型模式

- 原型模式实例与解析
  - 实例：邮件复制
    - 参考代码 (prototype)



# 原型模式

## 模式优缺点

### 原型模式的优点

- 当创建新的对象实例较为复杂时，使用原型模式可以简化对象的创建过程，通过一个已有实例可以提高新实例的创建效率
- 可以动态增加或减少产品类
- 原型模式提供了简化的创建结构

# 原型模式

- 模式优缺点
  - 原型模式的缺点
    - 需要为每一个类配备一个克隆方法，而且这个克隆方法需要对类的功能进行通盘考虑，这对全新的类来说不是很难，但对已有的类进行改造时，不一定是件容易的事，必须修改其源代码，违背了“开闭原则”

# 原型模式

- 模式应用
  - 很多软件提供的复制（Ctrl+C）和粘贴（Ctrl+V）操作就是原型模式的应用，复制得到的对象与原型对象是两个类型相同但内存地址不同的对象，通过原型模式可以大大提高对象的创建效率。



Copy and Paste 之母 Evelyn Berezin

☺☺☺☺☺☺

# 创建型模式

- 创建型模式简介
  - 简单工厂模式（Simple Factory）
  - 工厂方法模式（Factory Method）
  - 抽象工厂模式（Abstract Factory）
  - 建造者模式（Builder）
  - 原型模式（Prototype）
  - 单例模式（Singleton）



# 单例模式

## 模式动机

- 对于系统中的某些类来说，只有一个实例很重要，例如，一个系统中可以存在多个打印任务，但是只能有一个正在工作的任务；一个系统只能有一个窗口管理器或文件系统；一个系统只能有一个计时工具或ID（序号）生成器




真假美猴王



# 单例模式

## 模式动机

- 如何保证一个类只有一个实例并且这个实例易于被访问呢？定义一个全局变量可以确保对象随时都可以被访问，但不能防止我们实例化多个对象
- 一个更好的解决办法是让类自身负责保存它的唯一实例
- 这个类可以保证没有其他实例被创建，并且它可以提供一个访问该实例的方法。这就是单例模式的模式动机

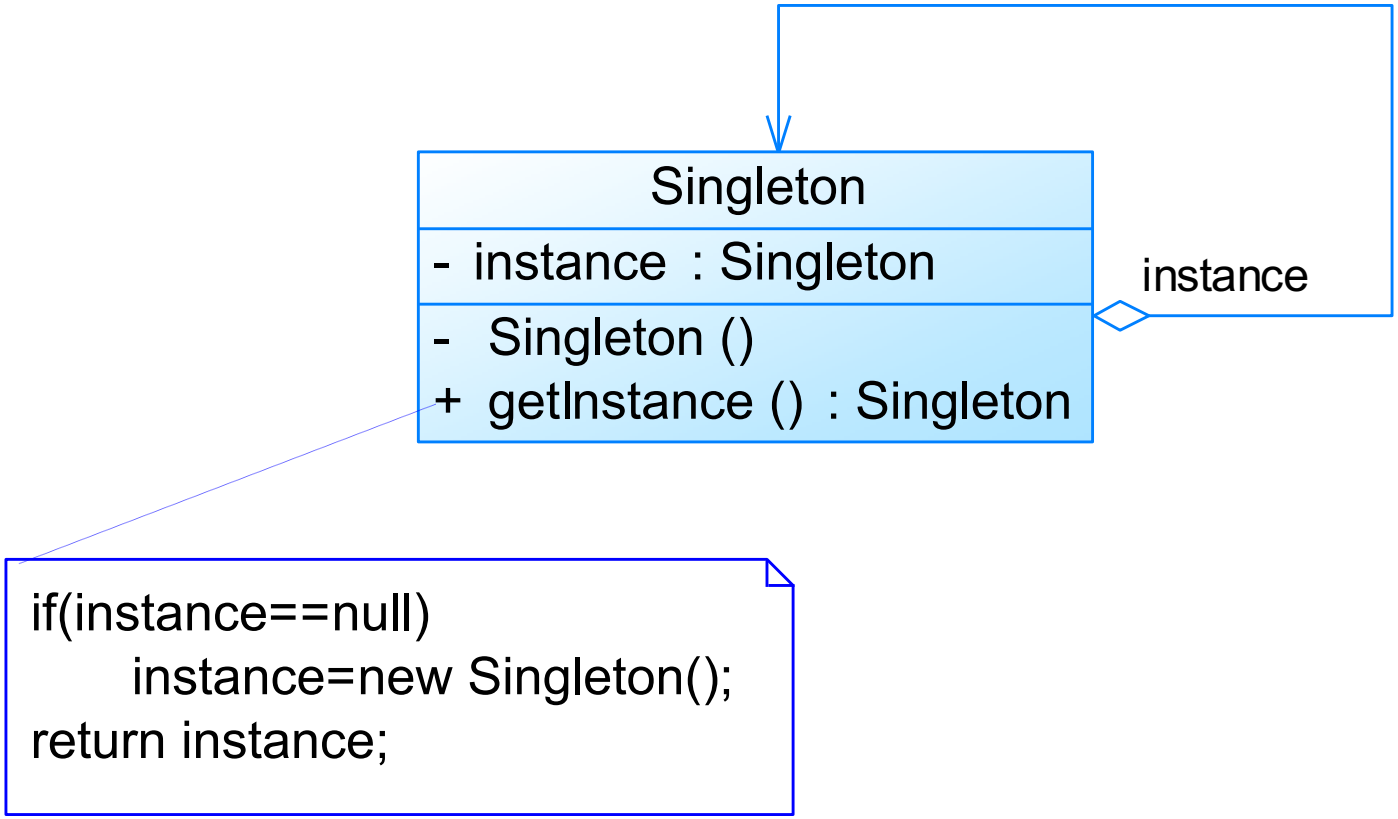
# 单例模式

- 模式定义
  - 单例模式(Singleton Pattern)：单例模式确保某一个类只有一个实例，而且自行实例化并向整个系统提供这个实例，这个类称为单例类，它提供全局访问的方法
  - 单例模式的要点有三个：一是某个类只能有一个实例；二是它必须自行创建这个实例；三是它必须自行向整个系统提供这个实例
  - 单例模式是一种创建型模式

# 单例模式

单例模式包含如下角色
- Singleton：单例

## 模式结构



# 单例模式

- 模式分析
  - 单例模式的目的是保证一个类仅有一个实例，并提供一个访问它的全局访问点。单例模式包含的角色只有一个，就是单例类——Singleton
  - 单例类拥有一个私有构造函数，确保用户无法通过new关键字直接实例化它
  - 除此之外，该模式中包含一个静态私有成员变量与静态公有的工厂方法，该工厂方法负责检验实例的存在性并实例化自己，然后存储在静态成员变量中，以确保只有一个实例被创建

# 单例模式

## 模式分析

- 单例模式的实现代码如下所示

```
public class Singleton {
    private static Singleton instance = null; //静态私有成员变量
    //私有构造函数
    private Singleton() {
    }
    //静态公有工厂方法，返回唯一实例
    public static Singleton getInstance() {
        if(instance == null)
            instance = new Singleton();
        return instance;
    }
}
```

# 单例模式

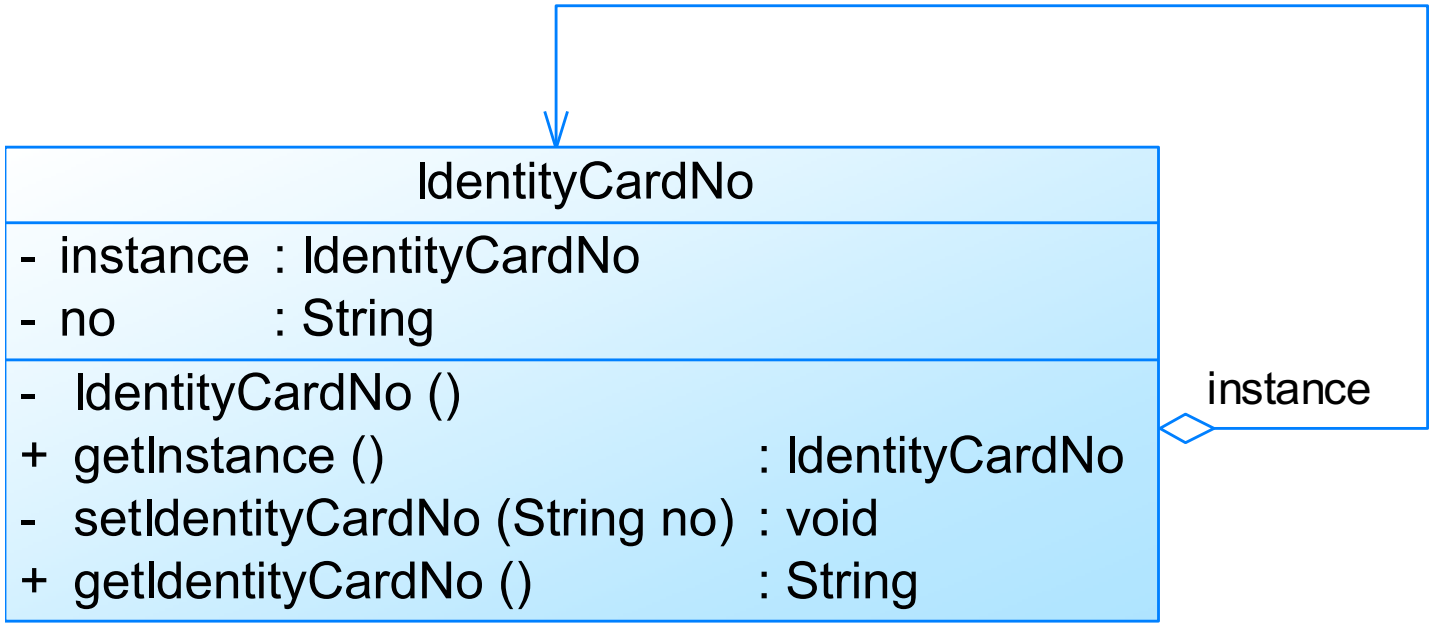
- 模式分析
  - 在单例模式的实现过程中，需要注意如下三点：
    - 单例类的构造函数为私有
    - 提供一个自身的静态私有成员变量
    - 提供一个公有的静态工厂方法

# 单例模式

- 单例模式实例与解析
  - 身份证号码
    - 在现实生活中，居民身份证号码具有唯一性，同一个人不允许有多个身份证号码，第一次申请身份证时将给居民分配一个身份证号码，如果之后因为遗失等原因补办时，还是使用原来的身份证号码，不会产生新的号码。现使用单例模式模拟该场景

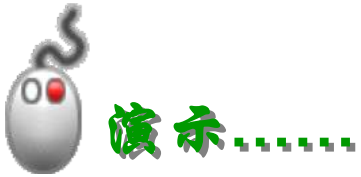
# 单例模式

- 单例模式实例与解析
  - 实例一：身份证号码



# 单例模式

- 单例模式实例与解析
  - 实例一：身份证号码
    - 参考代码 (singleton)



# 单例模式

## 模式优缺点

### 单例模式的优点

- 提供了对唯一实例的受控访问。因为单例类封装了它的唯一实例，所以它可以严格控制客户怎样以及何时访问它，并为设计及开发团队提供了共享的概念。
- 由于在系统内存中只存在一个对象，因此可以节约系统资源，对于一些需要频繁创建和销毁的对象，单例模式无疑可以提高系统的性能。
- 允许可变数目的实例。我们可以基于单例模式进行扩展，使用与单例控制相似的方法来获得指定个数的对象实例。

# 单例模式

## 模式优缺点

### 单例模式的缺点

- 由于单例模式中没有抽象层，因此单例类的扩展有很大的困难。
- 单例类的职责过重，在一定程度上违背了“单一职责原则”。因为单例类既充当了工厂角色，提供了工厂方法，同时又充当了产品角色，包含一些业务方法，将产品的创建和产品的本身的功能融合到一起。
- 滥用单例将带来一些负面问题，如为了节省资源将数据库连接池对象设计为单例类，可能会导致共享连接池对象的程序过多而出现连接池溢出；现在很多面向对象语言(如Java、C#)的运行环境都提供了自动垃圾回收的技术，因此，如果实例化的对象长时间不被利用，系统会认为它是垃圾，会自动销毁并回收资源，下次利用时又将重新实例化，这将导致对象状态的丢失。

# 单例模式

## 模式适用环境

### 在以下情况下可以使用单例模式：

- 系统只需要一个实例对象，如系统要求提供一个唯一的序列号生成器，或者需要考虑资源消耗太大而只允许创建一个对象。
- 客户调用类的单个实例只允许使用一个公共访问点，除了该公共访问点，不能通过其他途径访问该实例。
- 在一个系统中要求一个类只有一个实例时才应当使用单例模式。反过来，如果一个类可以有几个实例共存，就需要对单例模式进行改进，使之成为多例模式。

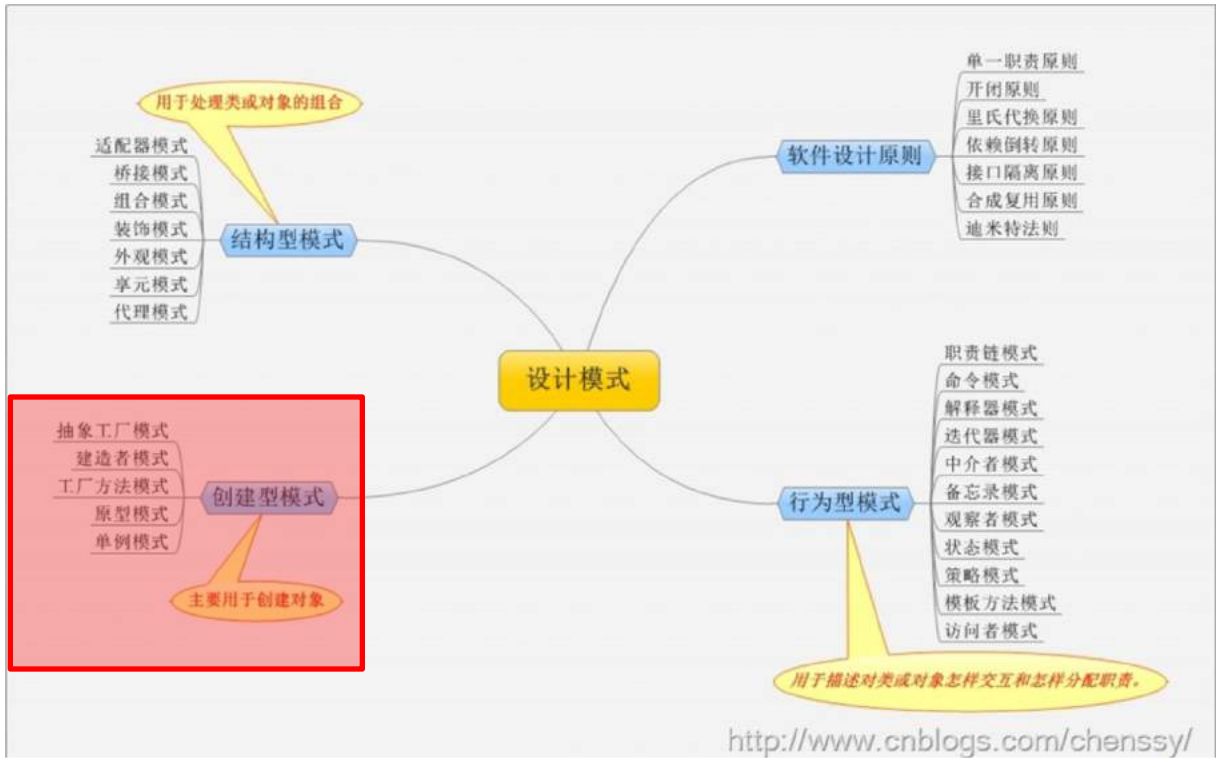
# 单例模式

- 模式应用
  - java.lang.Runtime类

```
public class Runtime {
    private static Runtime currentRuntime = new
Runtime();
    public static Runtime getRuntime() {
        return currentRuntime;
    }
    private Runtime() {}
}
```

# 本章教学内容

- 创建型设计模式
  - 简单工厂模式
  - 工厂方法模式
  - 抽象工厂模式
  - 建造者模式
  - 原型模式
  - 单例模式



# 创建型模式

- 概述
  - 创建型模式（Creational Pattern）对类的实例化过程进行抽象，将软件模块中对象的创建和对象的使用分离
  - 为了使软件的结构更加清晰，外界对于这些对象只需要知道它们共同的接口，而不清楚其具体的实现细节，使整个系统的设计更加符合单一职责原则

# 创建型模式

- 概述
  - 创建型模式在创建什么（What），由谁创建（Who），何时创建（When）等方面都为软件设计者提供了尽可能大的灵活性
  - 创建型模式隐藏了类的实例的创建细节，通过隐藏对象如何被创建和组合在一起达到使整个系统独立的目的

# 设计模式

| 范围\目的 | 创建型模式 | 结构型模式 | 行为型模式 |
| --- | --- | --- | --- |
| **类模式** | 工厂方法模式 | （类）适配器模式 | 解释器模式<br>模板方法模式 |
| **对象模式** | 抽象工厂模式<br>建造者模式<br>原型模式<br>单例模式 | （对象）适配器模式<br>桥接模式<br>组合模式<br>装饰模式<br>外观模式<br>享元模式<br>代理模式 | 职责链模式<br>命令模式<br>迭代器模式<br>中介者模式<br>备忘录模式<br>观察者模式<br>状态模式<br>策略模式<br>访问者模式 |

第8章

# 创建型设计模式

陈龙彪
longbiaochen@xmu.edu.cn

厦门大学信息学院
2019年4月

# END






# Thanks!